

1.安装N8N搭建RAG+N8N+Supabase

- 1.1使用npm/yarn全局安装
- 1.2使用Docker
- 1.3从源代码运行

2.开始使用N8N

2.1目标：首先针对PDF实现向量数据库搭建

- 1. On from submission
- 2. Supabase Vector Store
- 3. Embeddings Ollama
- 4. Default Data Loader
- 5. Recursive Character Text Splitter

2.2目标：创建一个mcpserver，代替我们完成一些工作

- 1. MCP Server Trigger触发器节点
- 2. Answer questions with a vector store
- 3. Supabase Vector Store1
- 4. Embeddings Ollama1
- 5. Ollama Model

2.3目标：使用AI Agent，当交互时根据大模型理解任务要求，并执行对话内容

- 1. When chat message received
- 2. AI Agent
- 3. Ollama Chat Model
- 4. Simple Memory
- 5. MCP Client

2.4具体流程

Supabase实现向量数据库

云服务<https://supabase.com/>

本地部署

部署bug

openssl rand -hex 32 生成32字节即64字符的 Hex 密钥

Ngtok本地映射公网

Ollama下载向量化模型

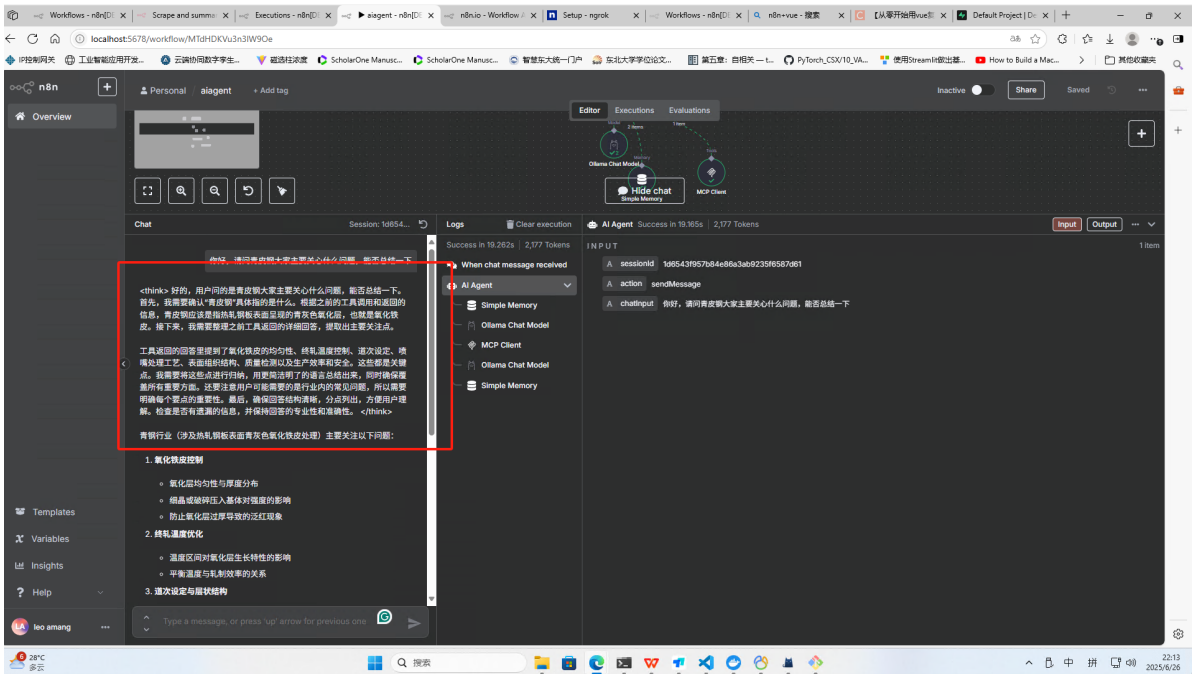
N8N支持的向量数据库

注：使用docker部署的地址localhost映射为host.docker.internal

N8n

n8n（发音为"n-eight-n"）是一个开源的工作流自动化工具，n8n提供了多种配置选项，可以通过环境变量或配置文件进行设置。相似的平台有coze和dify。

1.安装N8N搭建RAG+N8N+Supabase



安装n8n有三种方式：

1.1使用npm/yarn全局安装

使用npm/yarn全局安装

```
#使用npm全局安装n8n
npm install n8n -g

yarn global add n8n

n8n
```

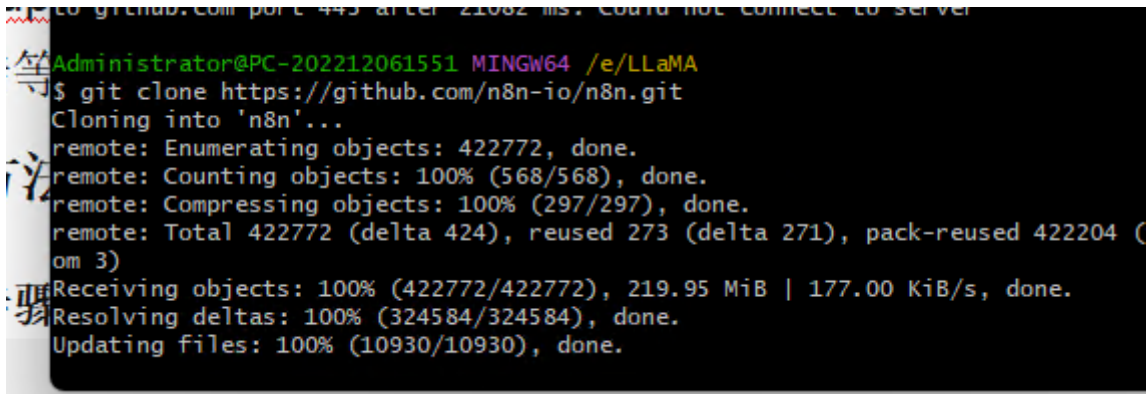
1.2使用Docker

这将在Docker容器中运行n8n，并将数据持久化到本地目录。

1.3从源代码运行

- 克隆仓库

```
git clone https://github.com/n8n-io/n8n.git
```



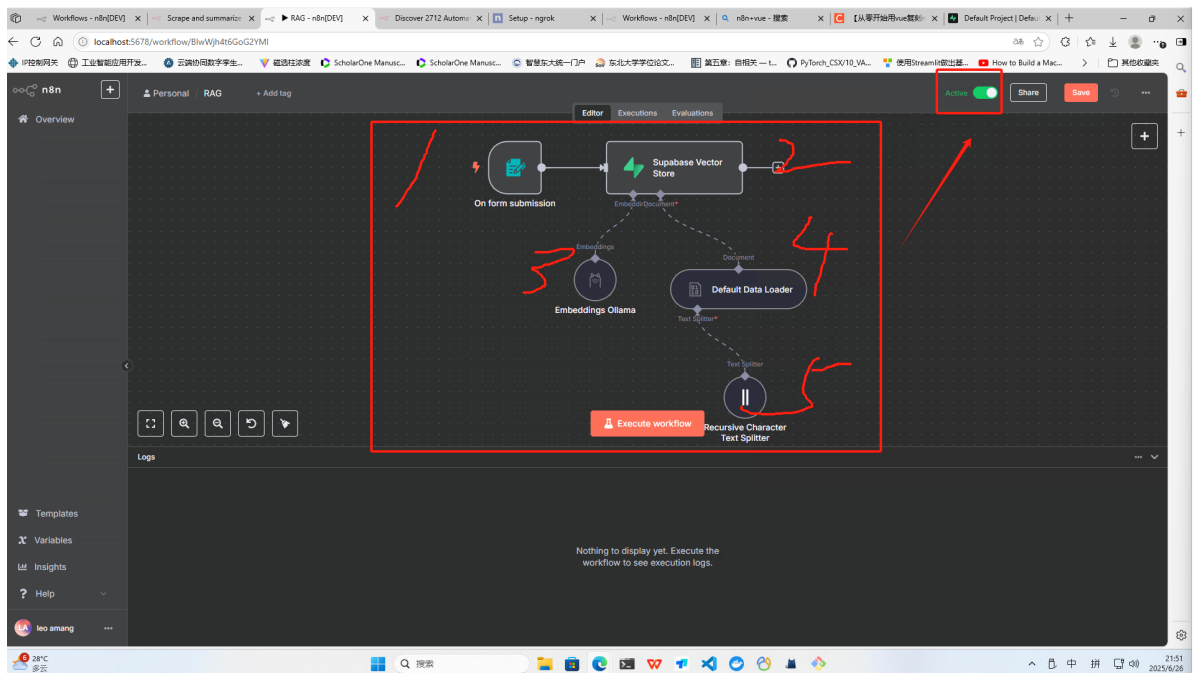
- 进入目录

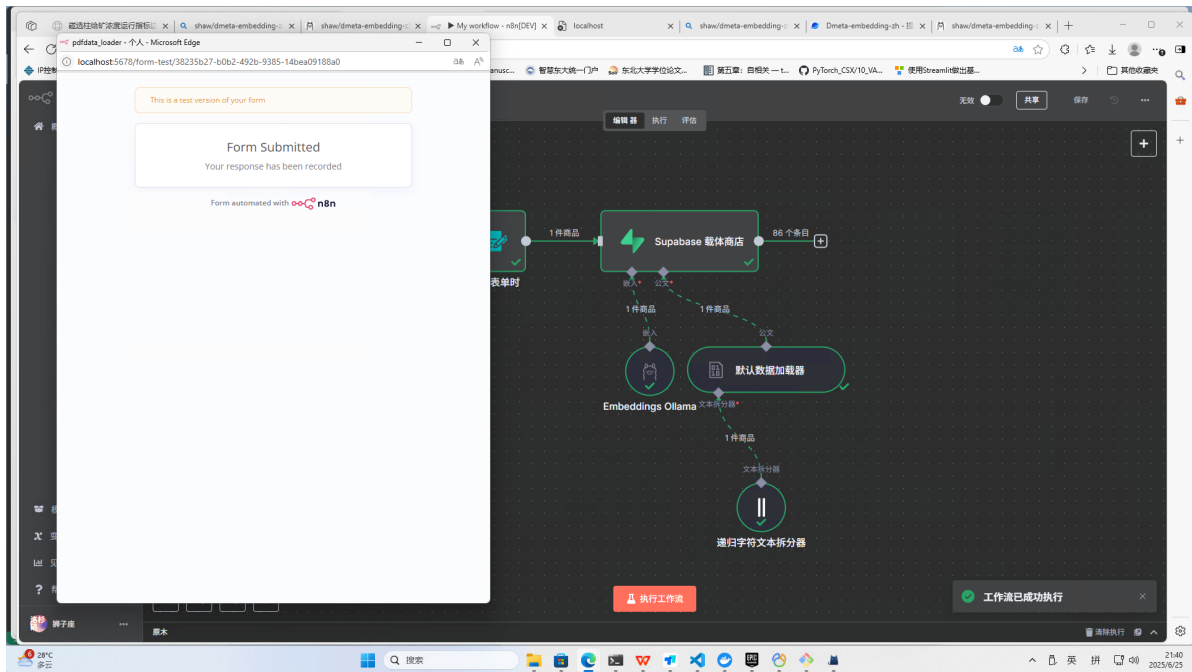
- 运行 n^8n

2.开始使用N8N

2.1目标：首先针对PDF实现向量数据库搭建

上传一个pdf文档，上传成功后可以在supabase服务器中看到保存的向量。完成后点击右上角的active启动工作流





1. On from submission

搜索n8 出现n8n表单，点击表单配置

按照下面的图片方式配置→测试一下，上传一个pdf文件→点击空白处完成配置

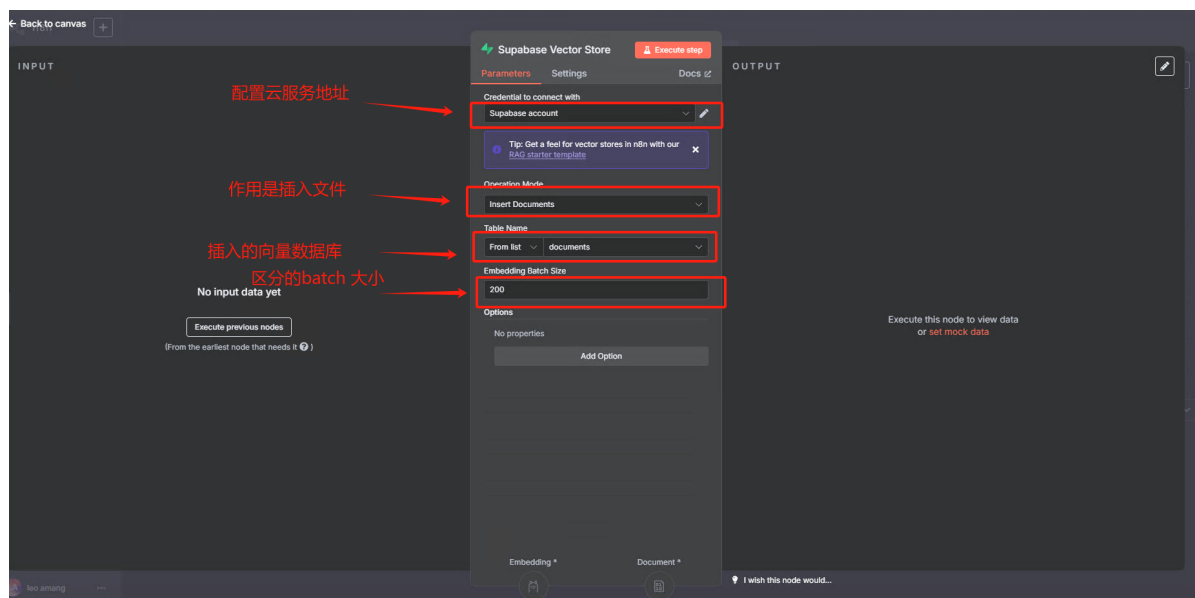


2. Supabase Vector Store

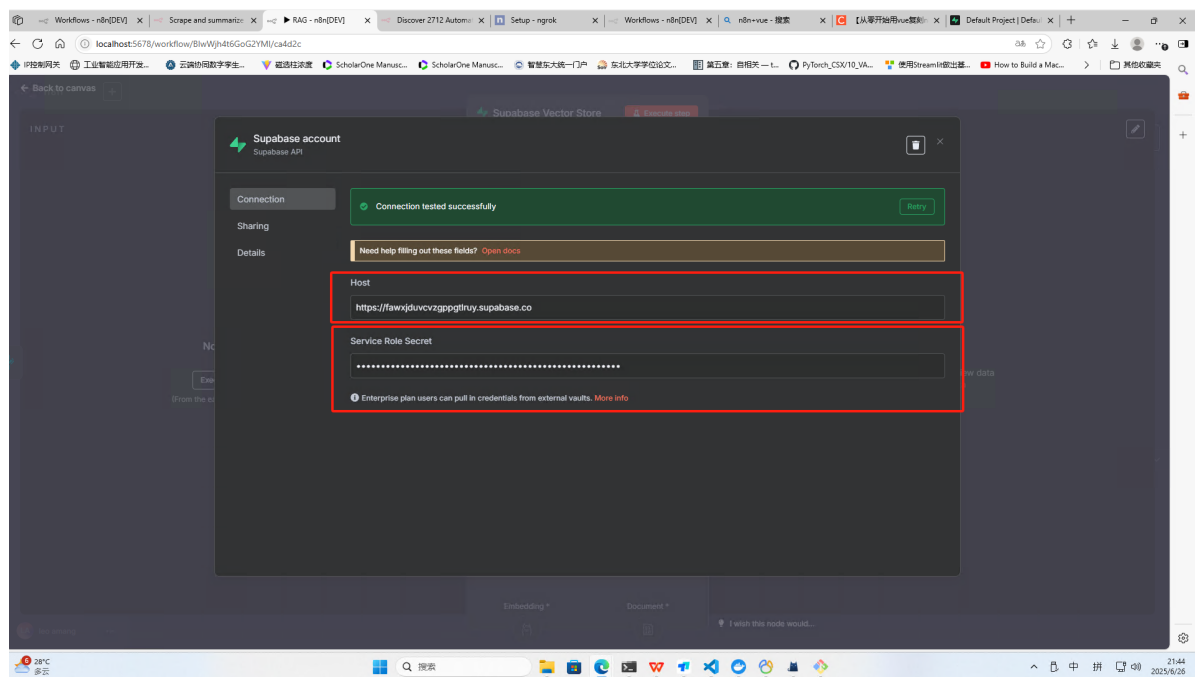
使用到supabase数据库创建向量数据表，可以使用supabase里面内置的一些文档向量工具，建立向量数据库，当向量转换大模型将pdf文件转换时，可以将转换后的向量存储到数据库。

supabase使用两种方式，一种是网页云服务器，一种是本地部署，看**Supabase实现向量数据库**

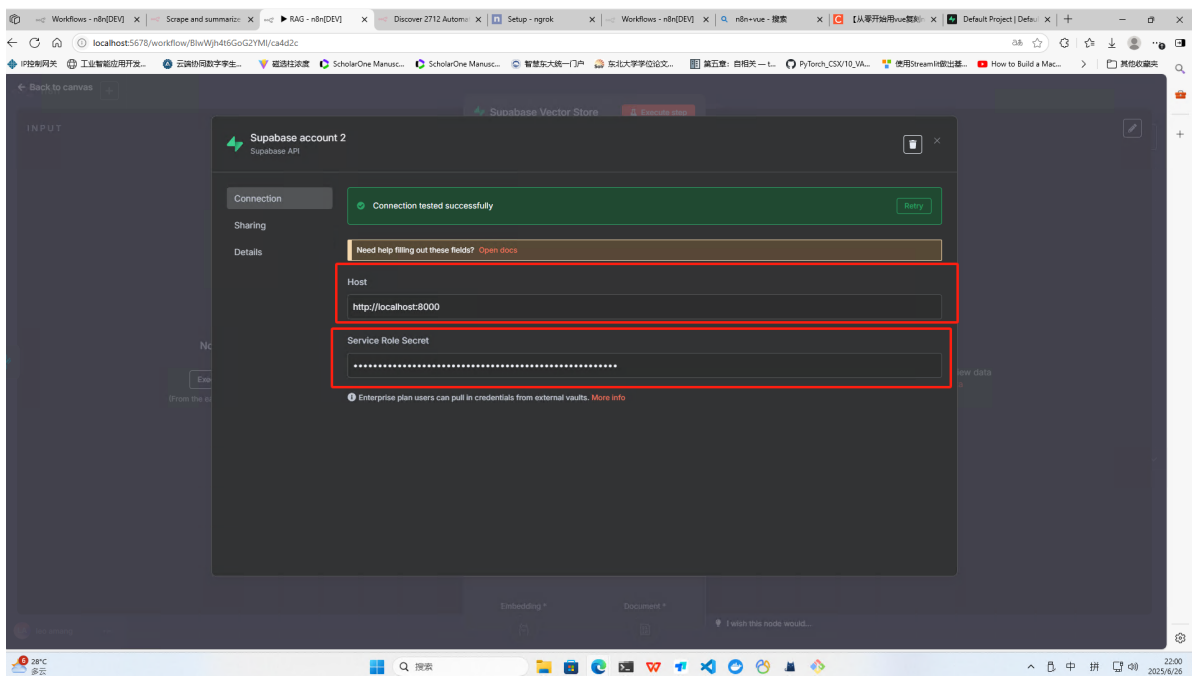
- batch size在进行向量化处理时，一次性向模型发送多少个数据分块，理解为批量处理能提高效率。



其中supabase的云配置如图

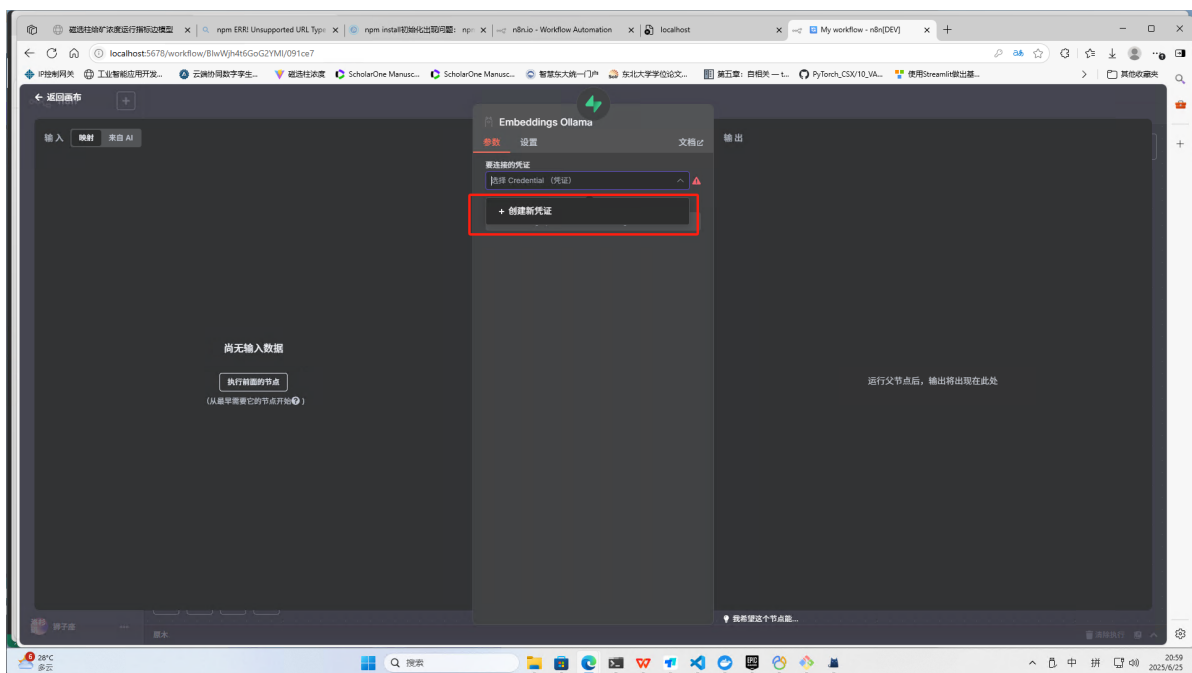


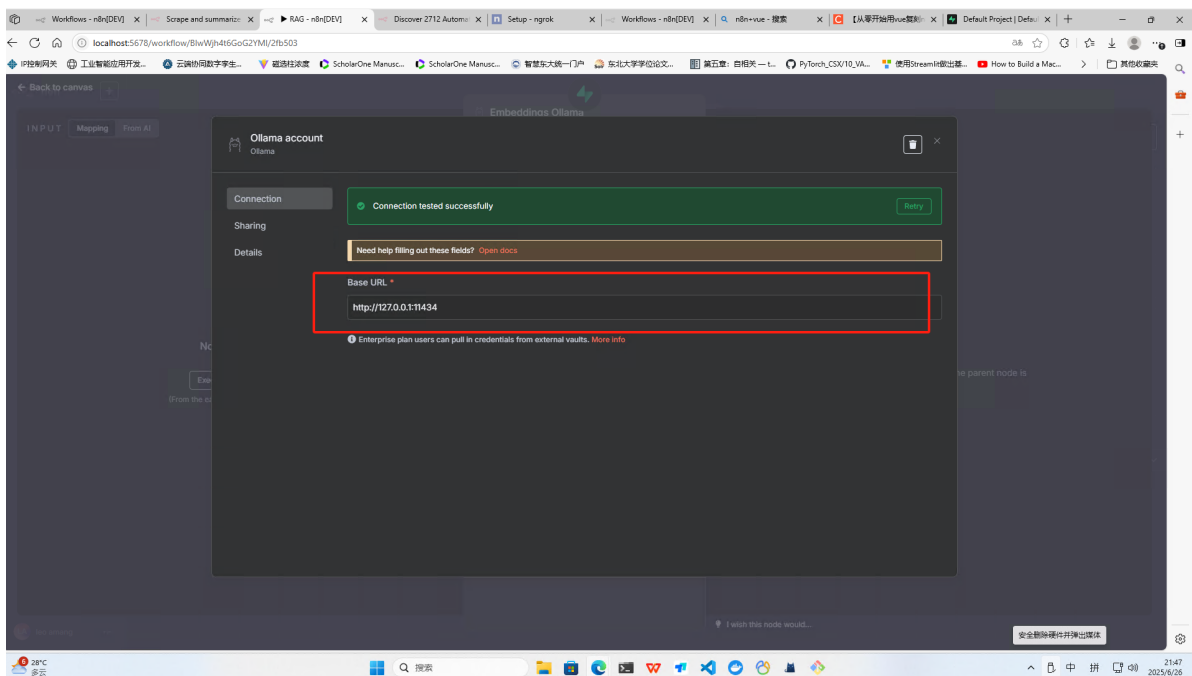
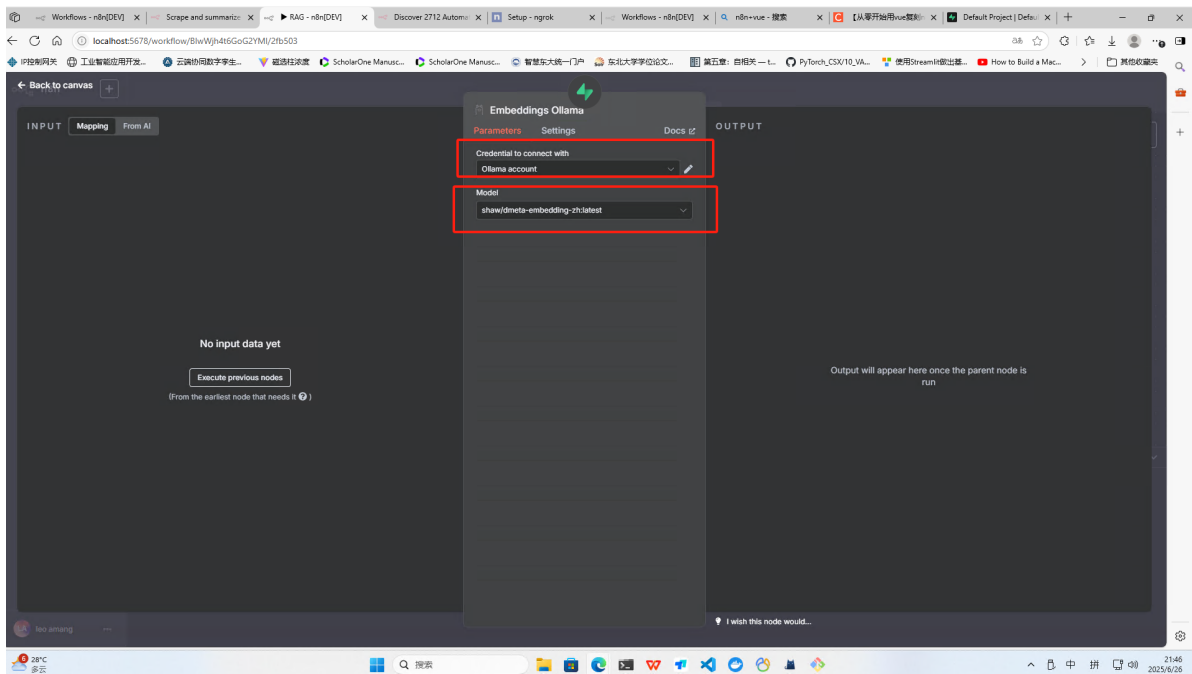
其中supabase的本地配置如图



3. Embeddings Ollama

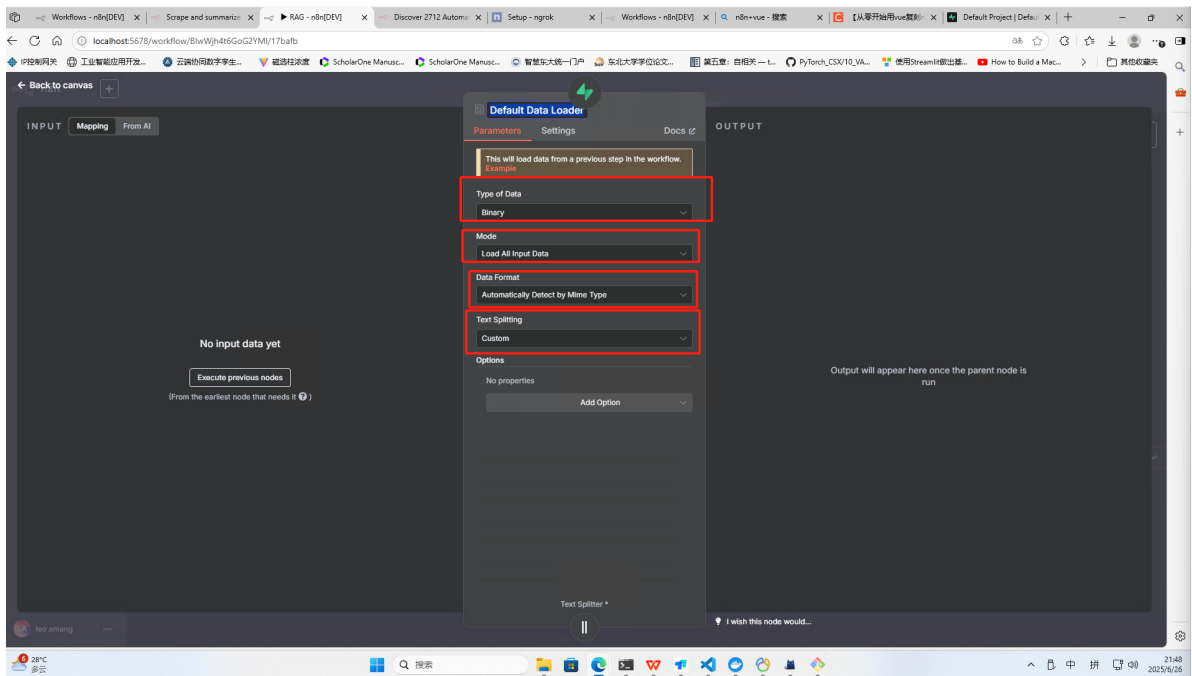
来自于ollama的向量库，ollama的配置和模型下载参考之前的文档。





4. Default Data Loader

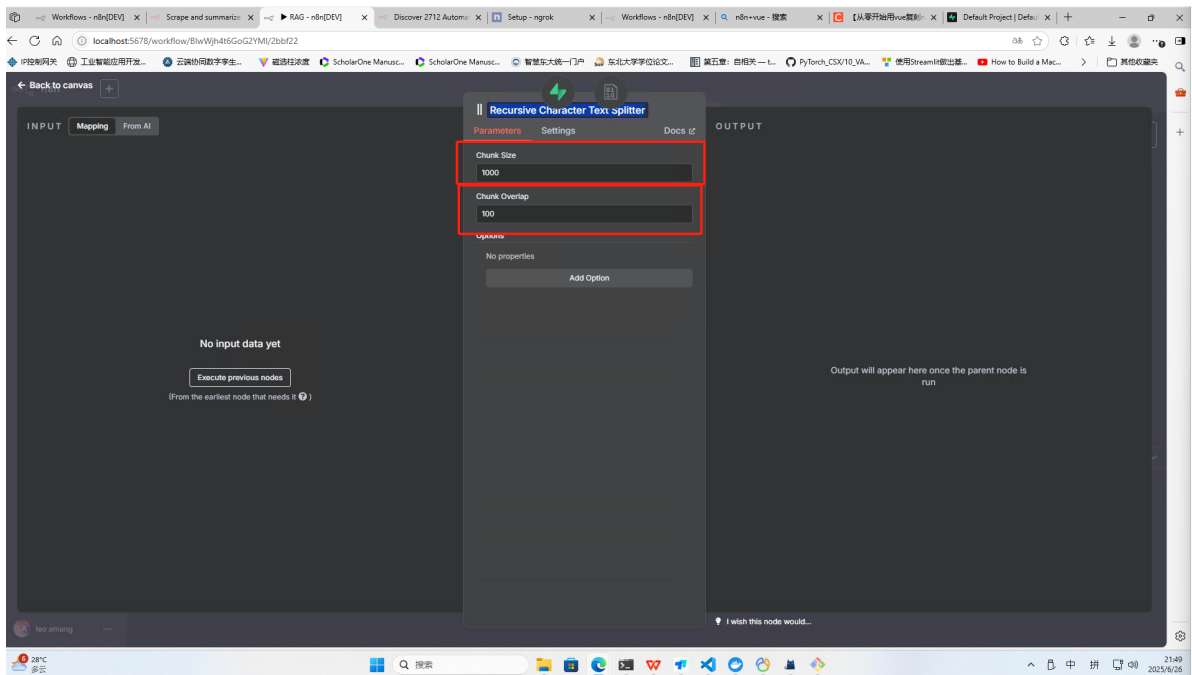
数据加载的默认方式，其中文本切块方式选择自定义custom



5. Recursive Character Text Splitter

检索特征文本切片，包括切片的大小1000，重叠部分的大小100

片段大包含不相关信息，太小可能包含信息不完整，所以使用1000,相邻的两个片段是重合,防止切割导致语义断裂问题

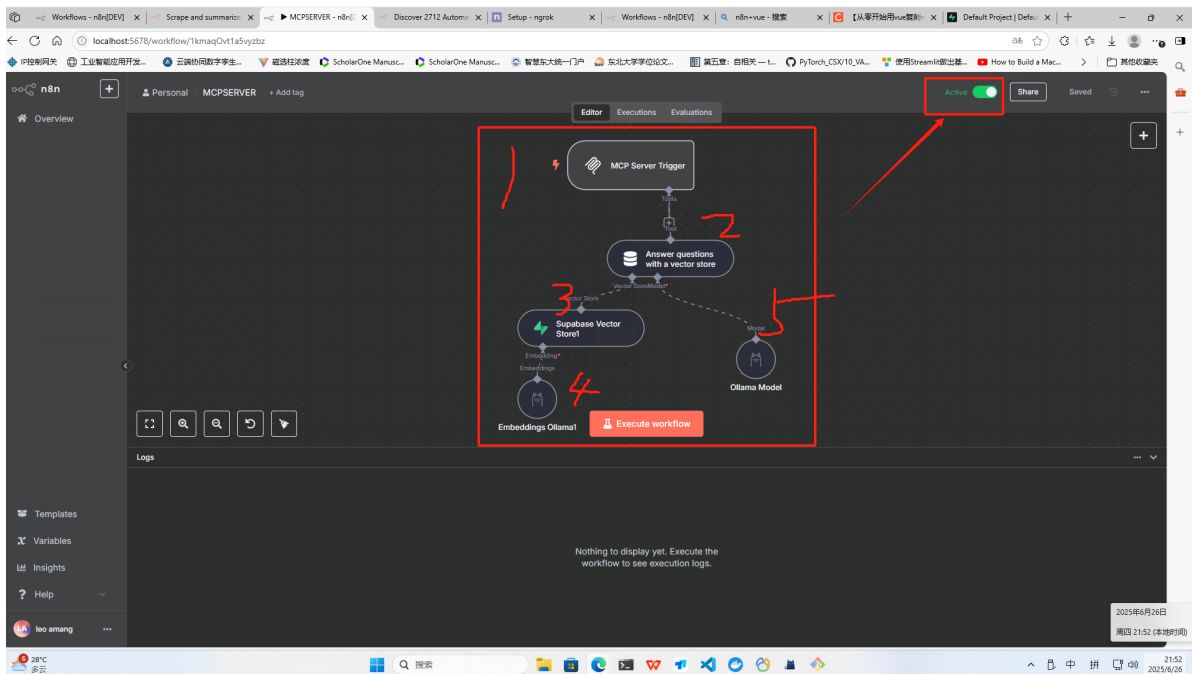


2.2目标：创建一个mcpserver，代替我们完成一些工作

重新一个工作区，按如下图连线，这个mcpserver工具的作用是整合我们后续连接的大模型和向量数据库

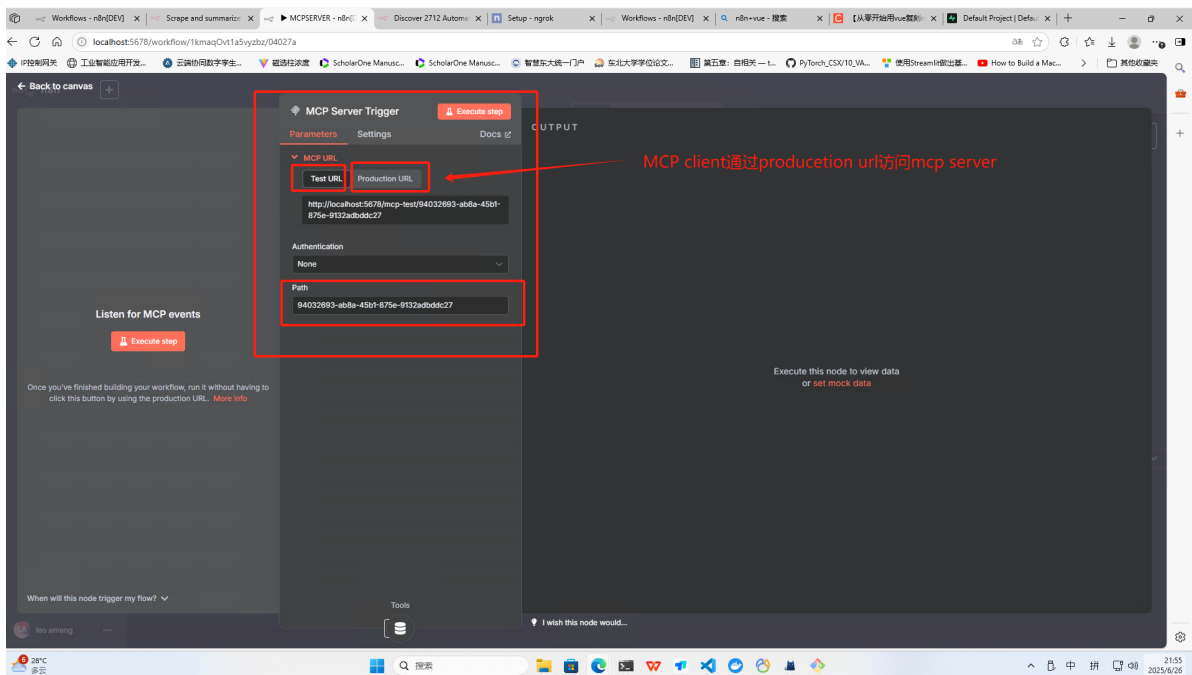
当mcp server接收到mcp client的请求信息时，会根据需求判断执行什么工作。

这里需要根据人问的问题寻求检索向量数据库最相似的4个答案，并将答案返回给mcp client，mcp给大模型参考然后回答问题。



1. MCP Server Trigger触发器节点

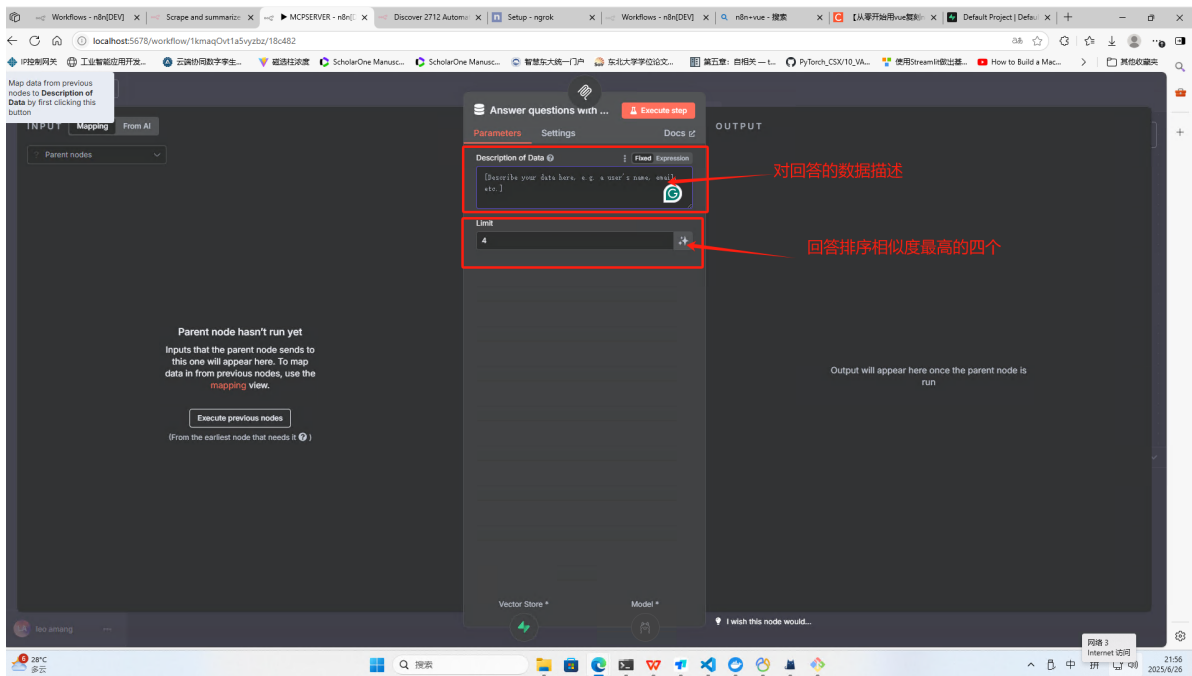
- mcp触发器节点可以让工作流可以变成一个接受外部请求对的服务，



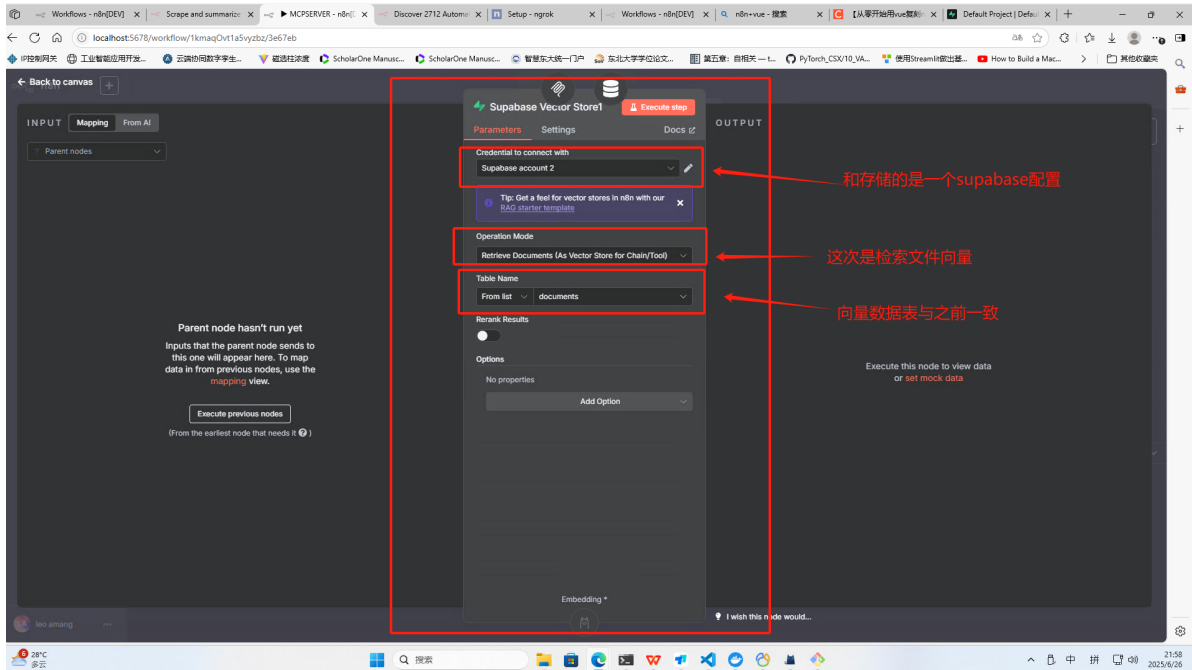
- 直接访问mcp的地址显示event: endpoint data: /mcp/94032693-ab8a-45b1-875e-9132adbddc27?sessionId=e81c0093-2c18-4819-a16f-e443f94251b8但是设置openai的时候连接不上
 - 直接访问 MCP 的地址返回的是一个 SSE (Server-Sent Events) 端点，而不是标准的 OpenAI API 兼容接口。说明MCP 不是以 OpenAI API 格式暴露服务，导致 OpenWebUI 无法直接连接。

2. Answer questions with a vector store

limit是指当我们提问时系统会从数据库找出多少个最相关的资料片段，作为参考信息提供给大模型，设置会直接影响回答的准确性和全面性。

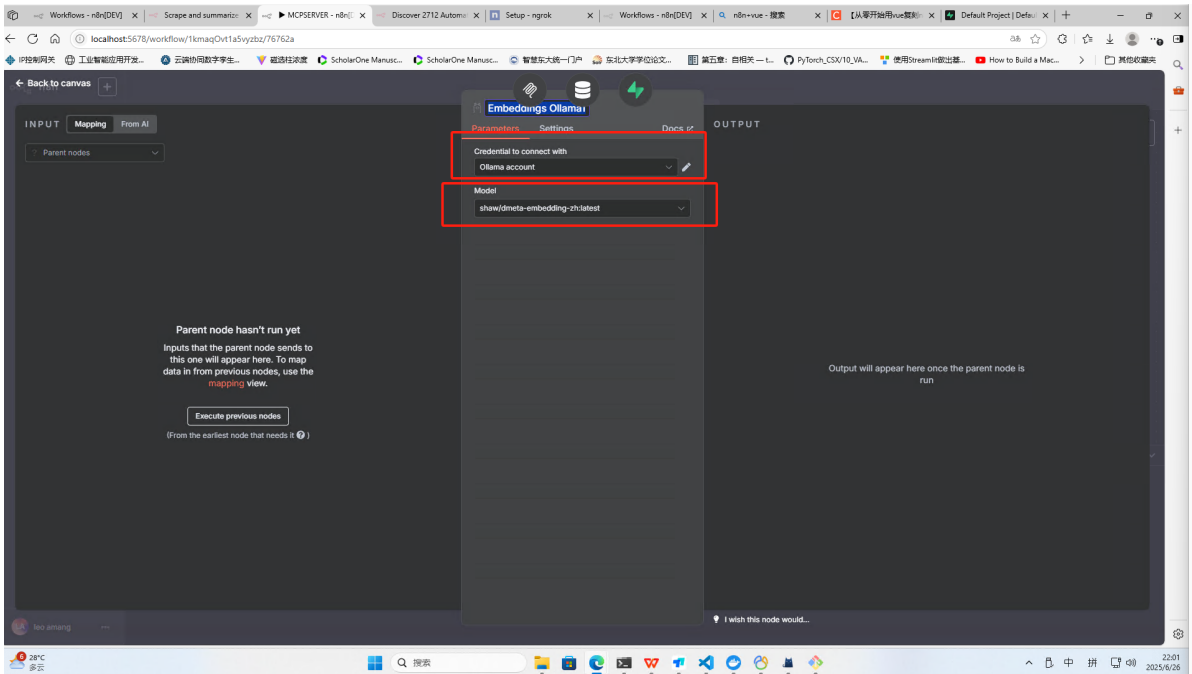


3. Supabase Vector Store1



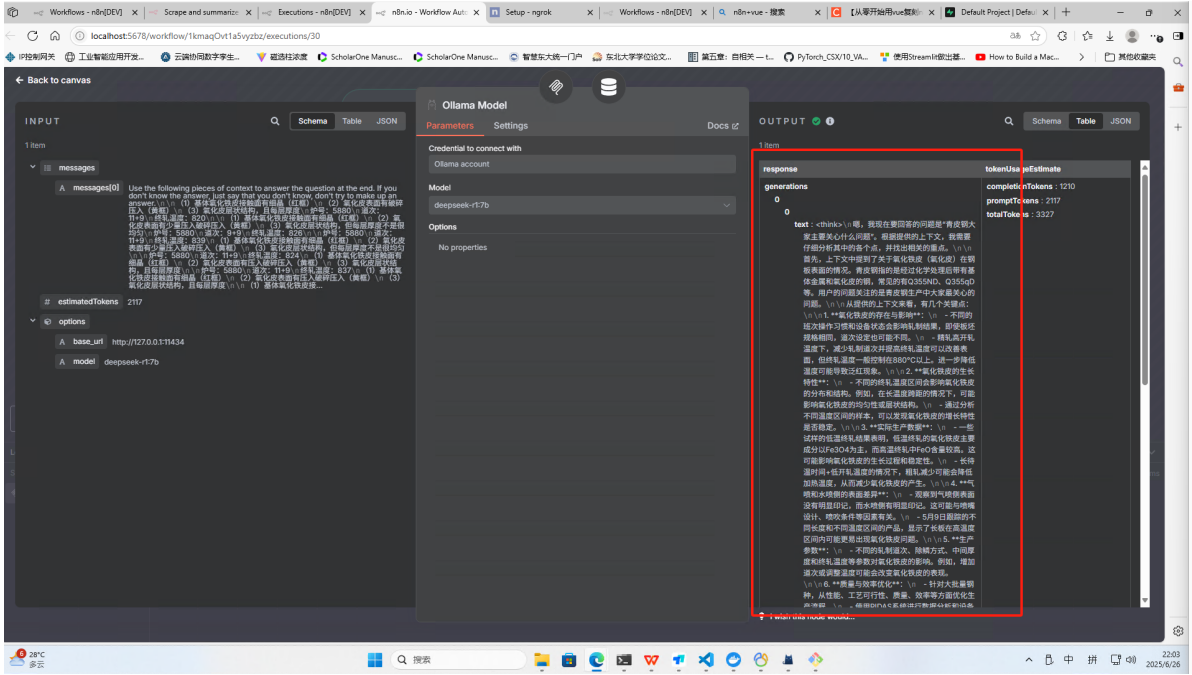
4. Embeddings Ollama1

这个向量大模型与之前存储的模型一致



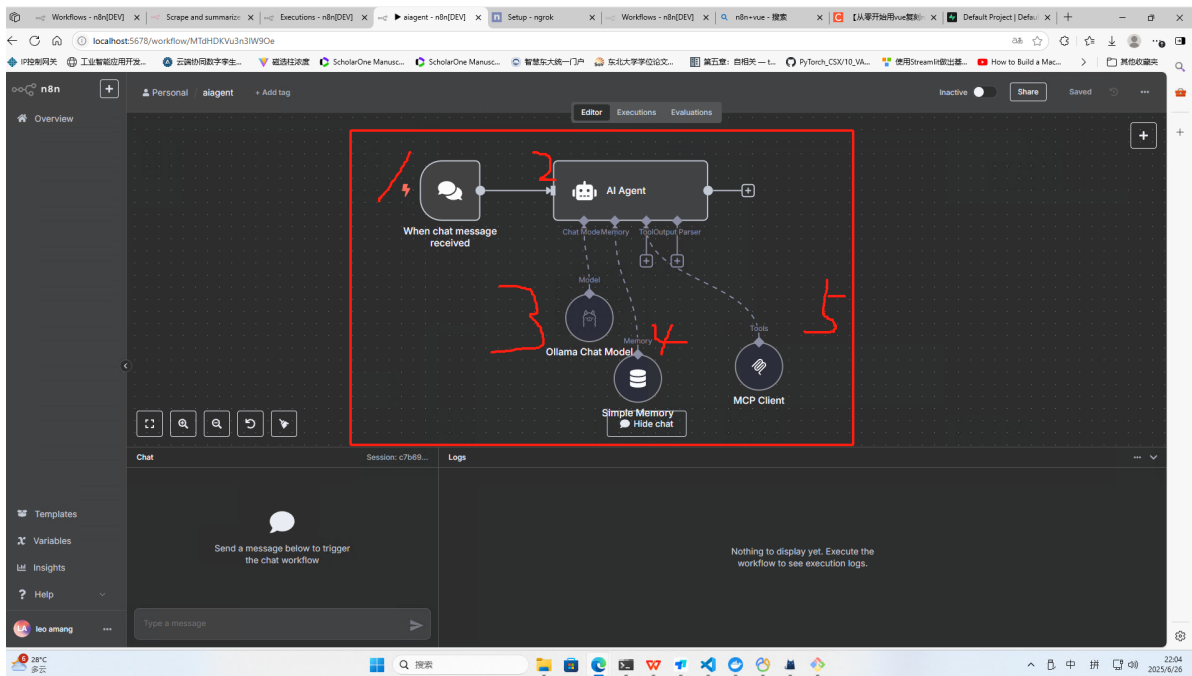
5. Ollama Model

这里配置大模型得到检索到的答案作为参考，回答问题



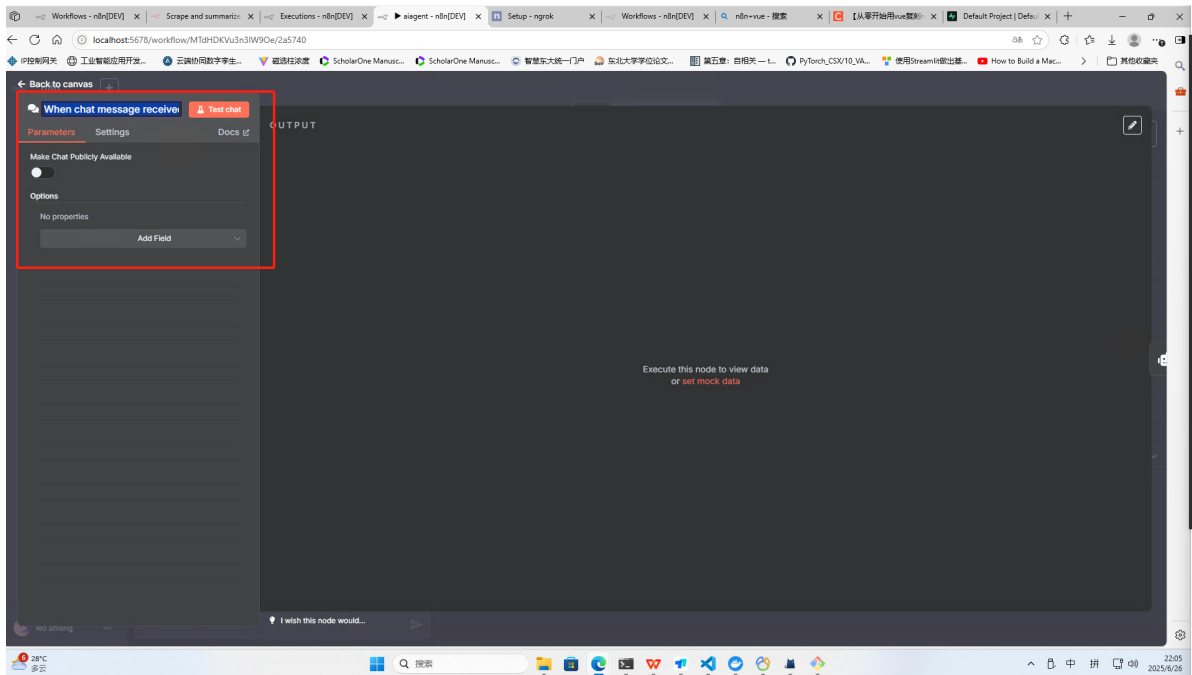
2.3目标：使用Agent，当交互时根据大模型理解任务要求，并执行对话内容

当交互时根据大模型理解任务要求，获取问题，并将问题返回给chat界面



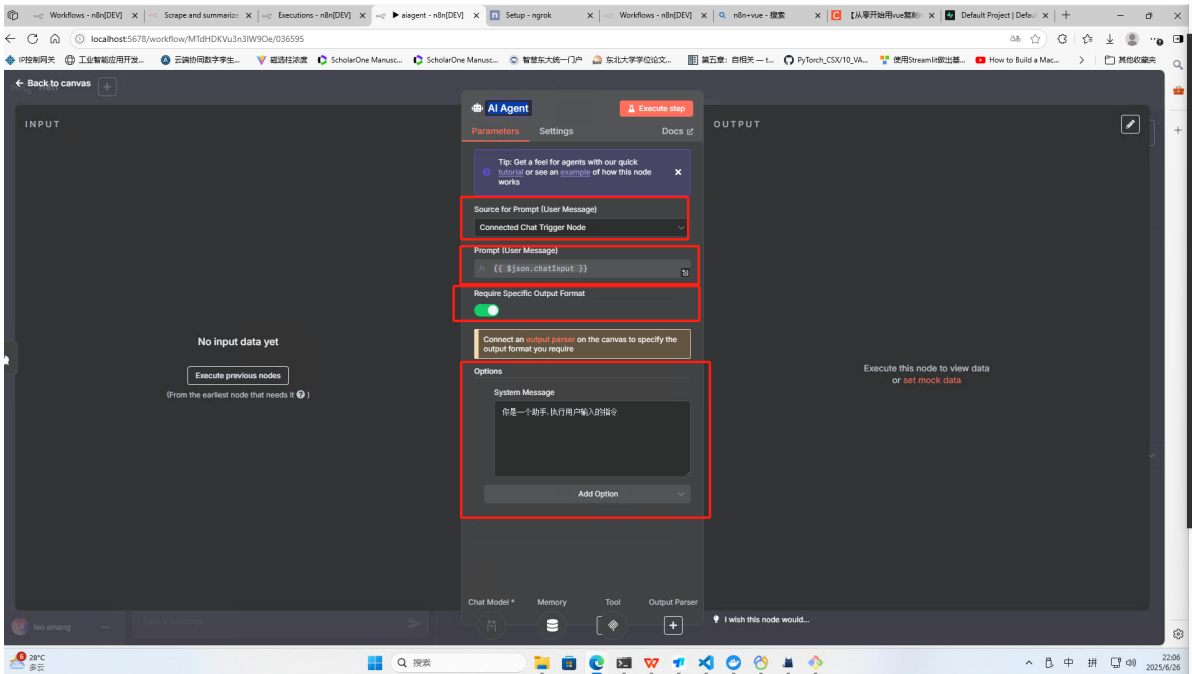
1. When chat message received

chat触发节点



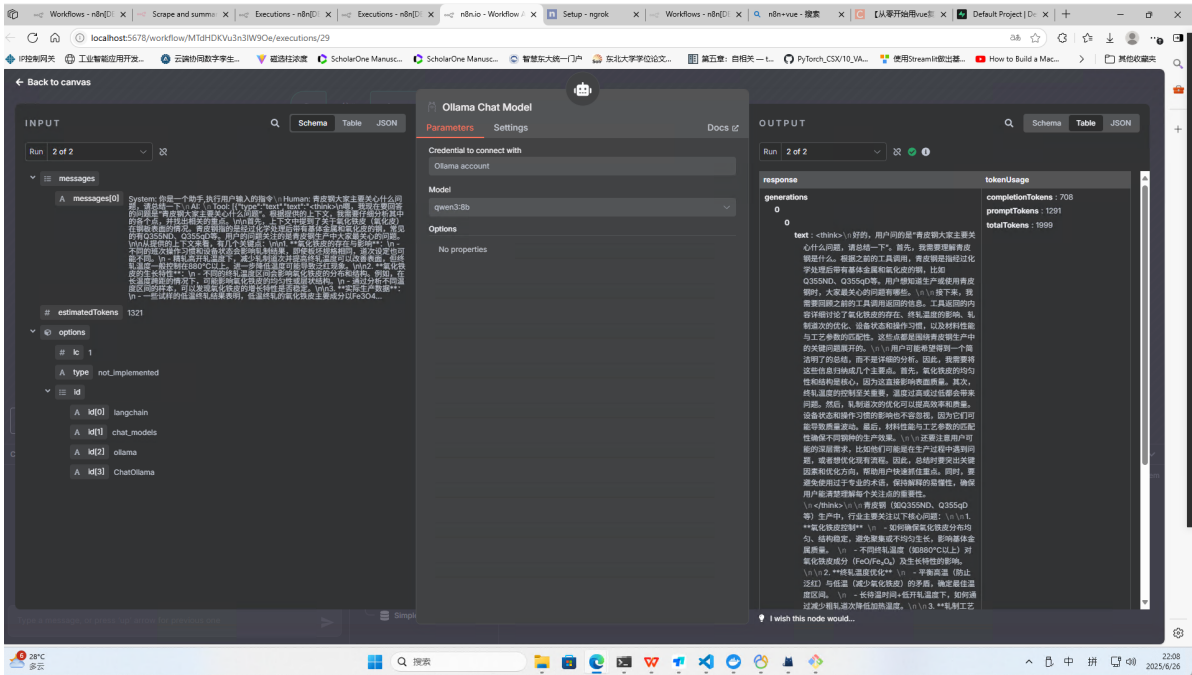
2. AI Agent

可以配置一下系统信息



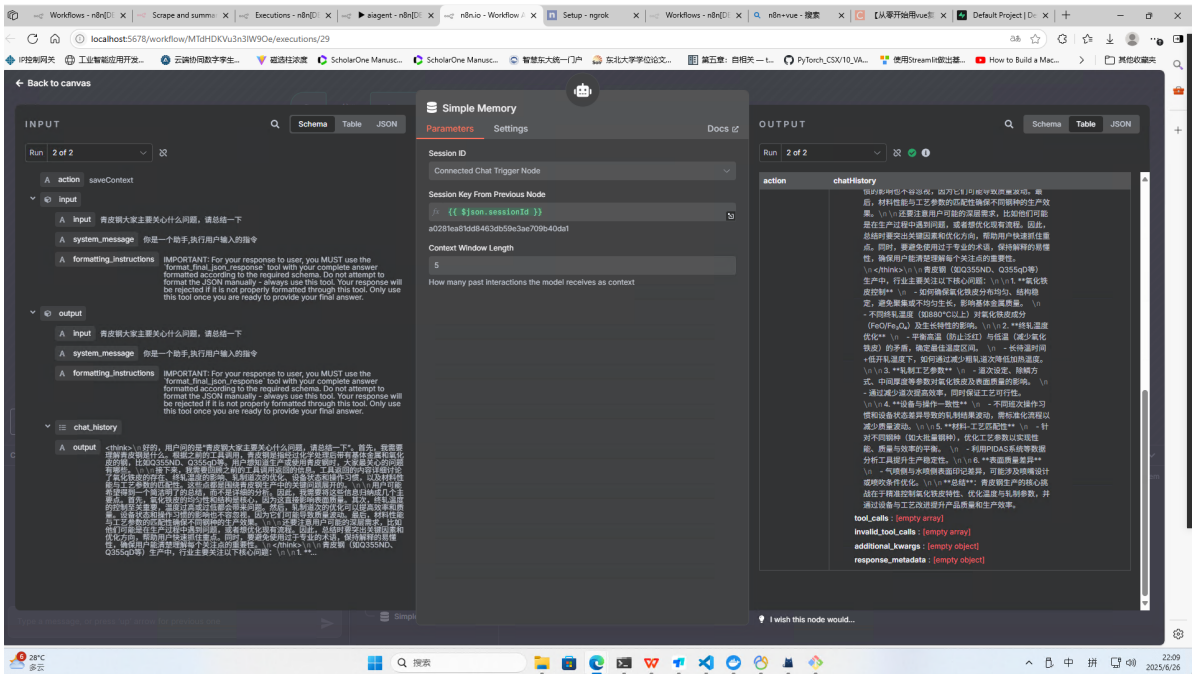
3. Ollama Chat Model

利用大模型理解用户需求，获取mcp server大模型传过来的信息进行重新总结



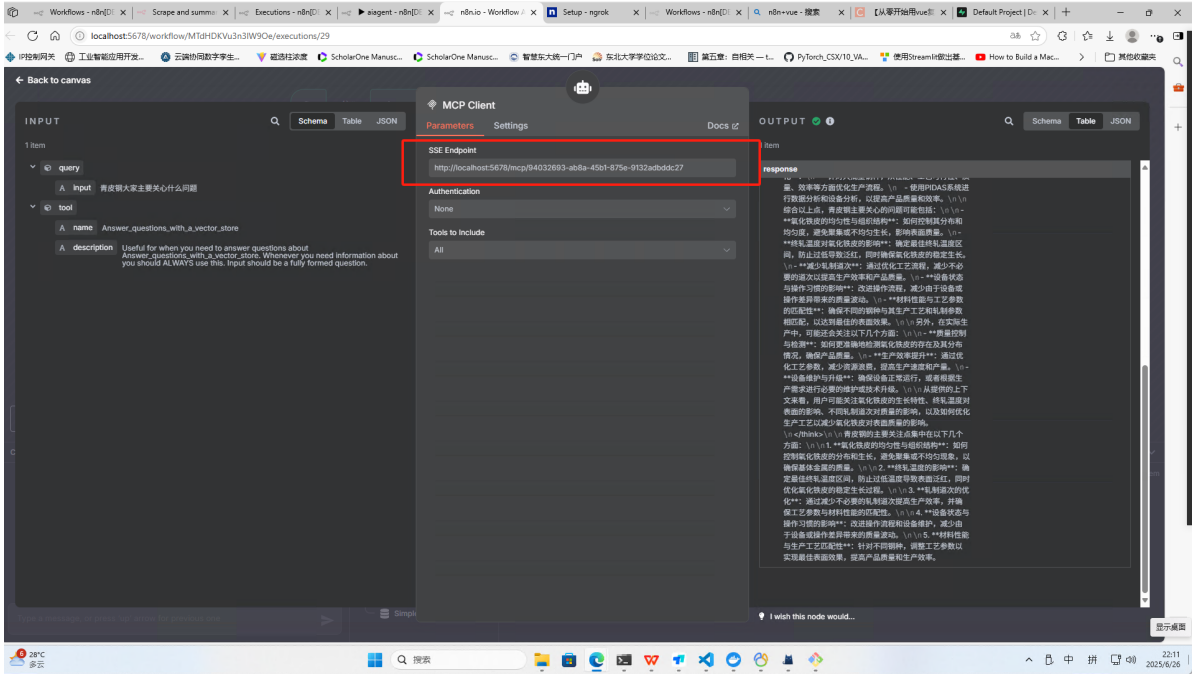
4. Simple Memory

记忆，历史回溯，记住之前输入的问题和回答



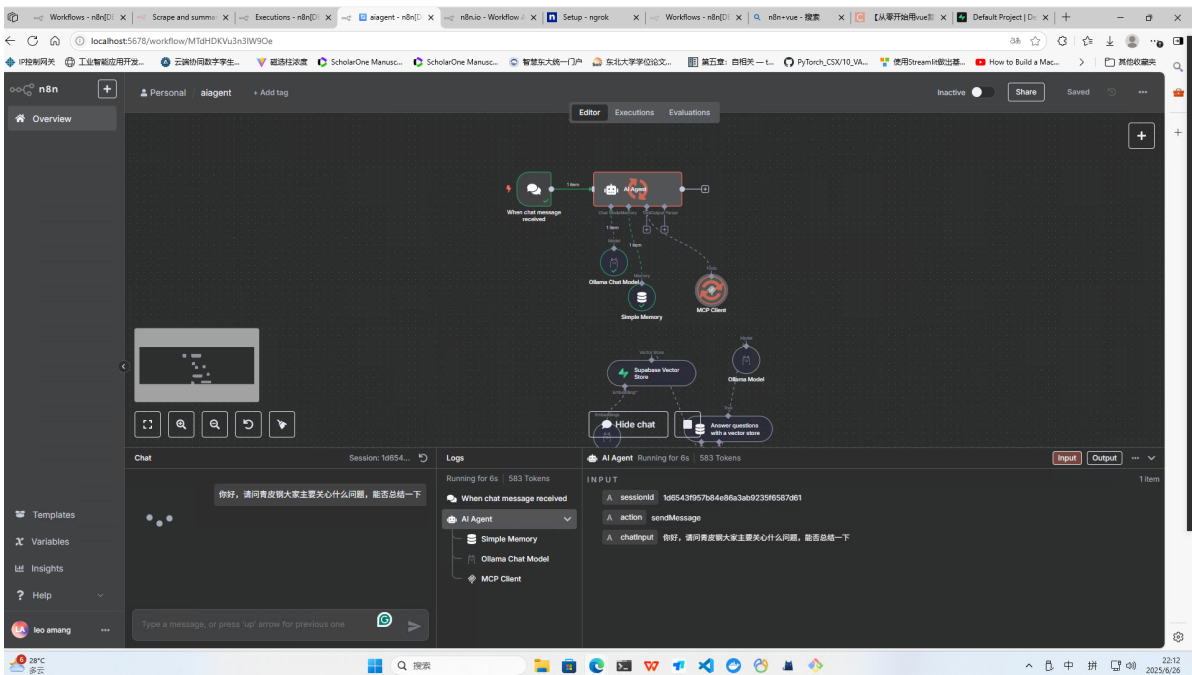
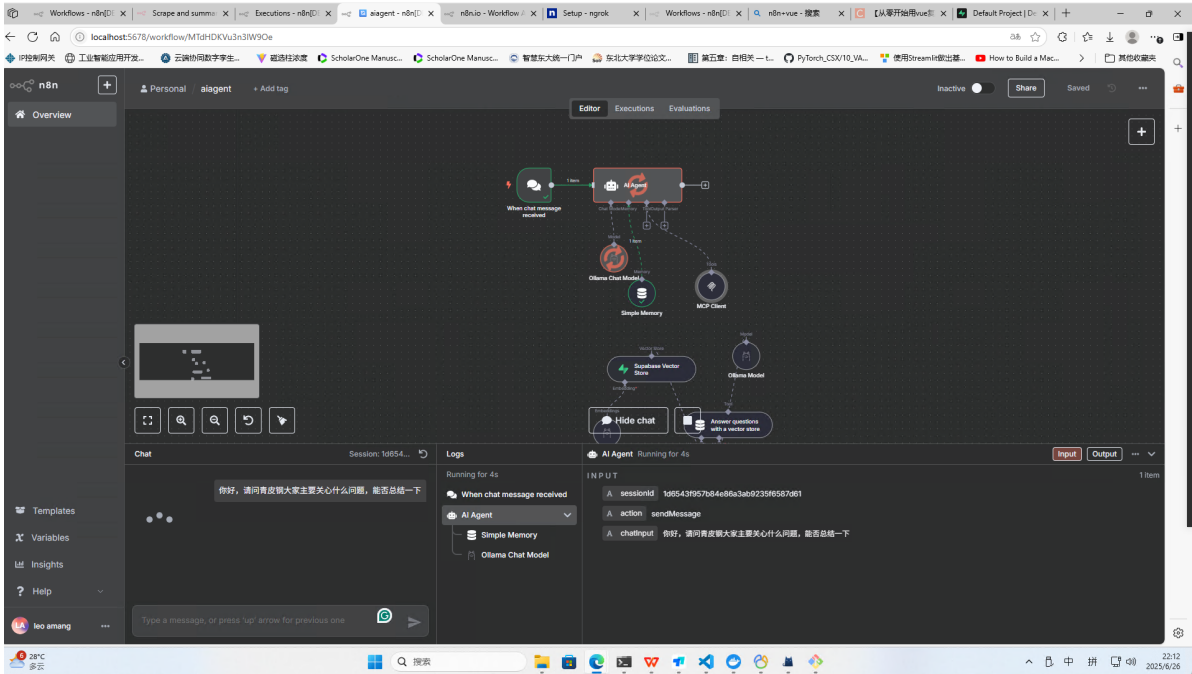
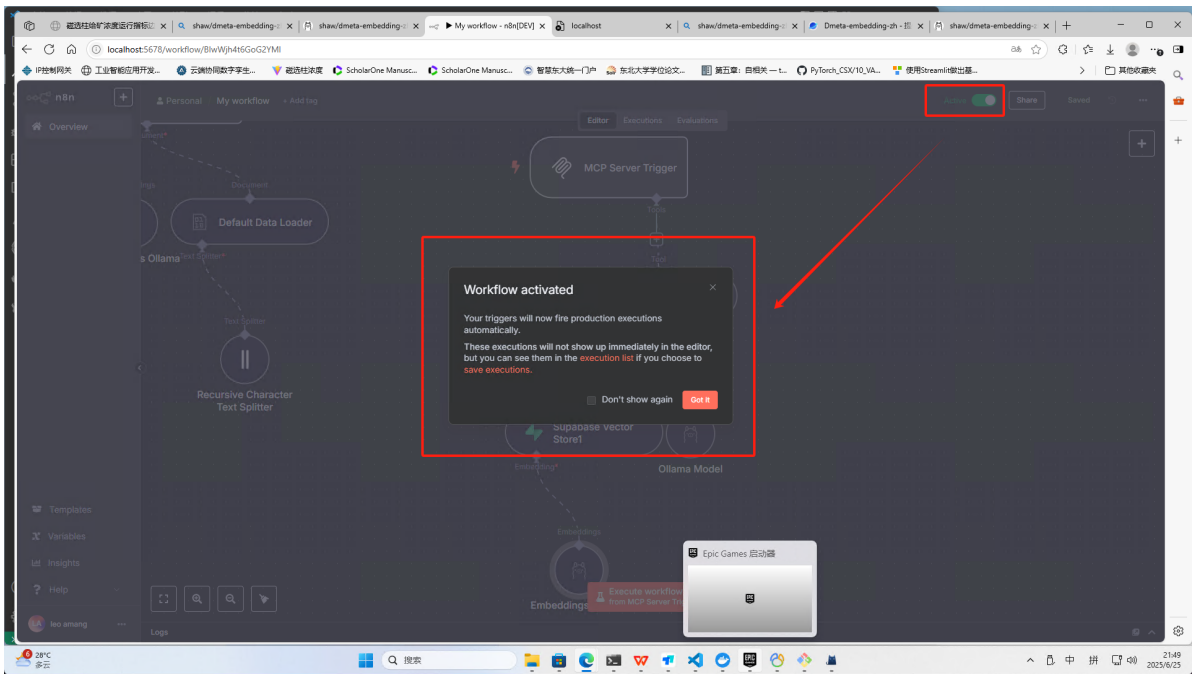
5. MCP Client

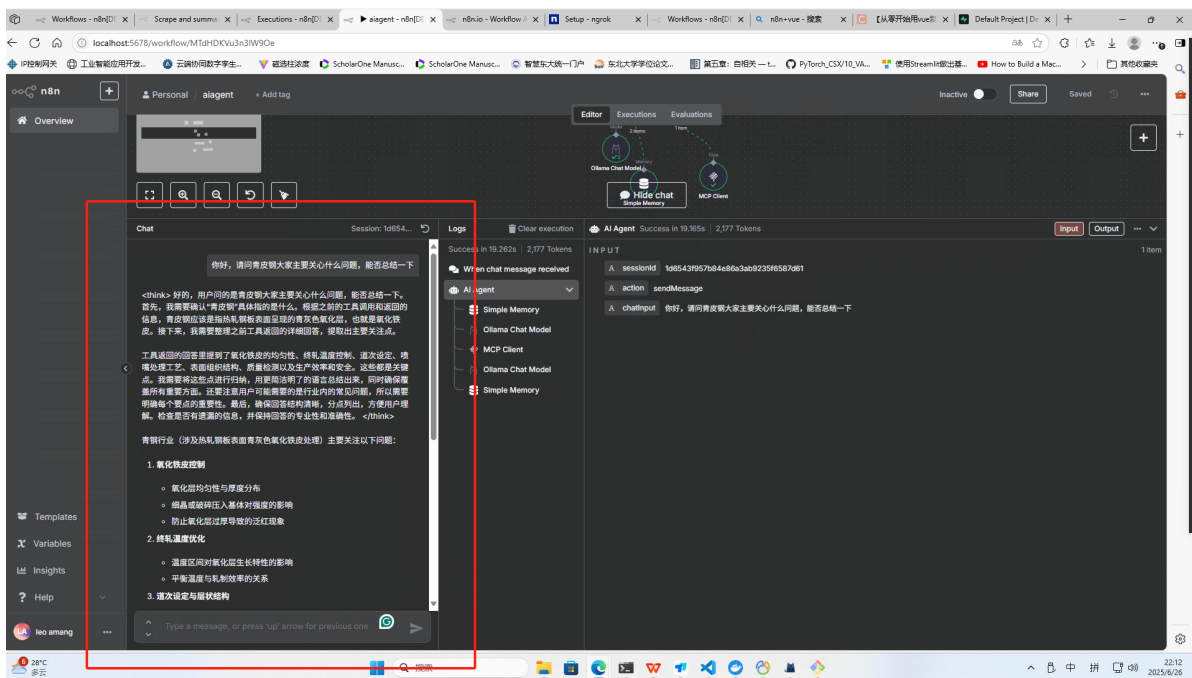
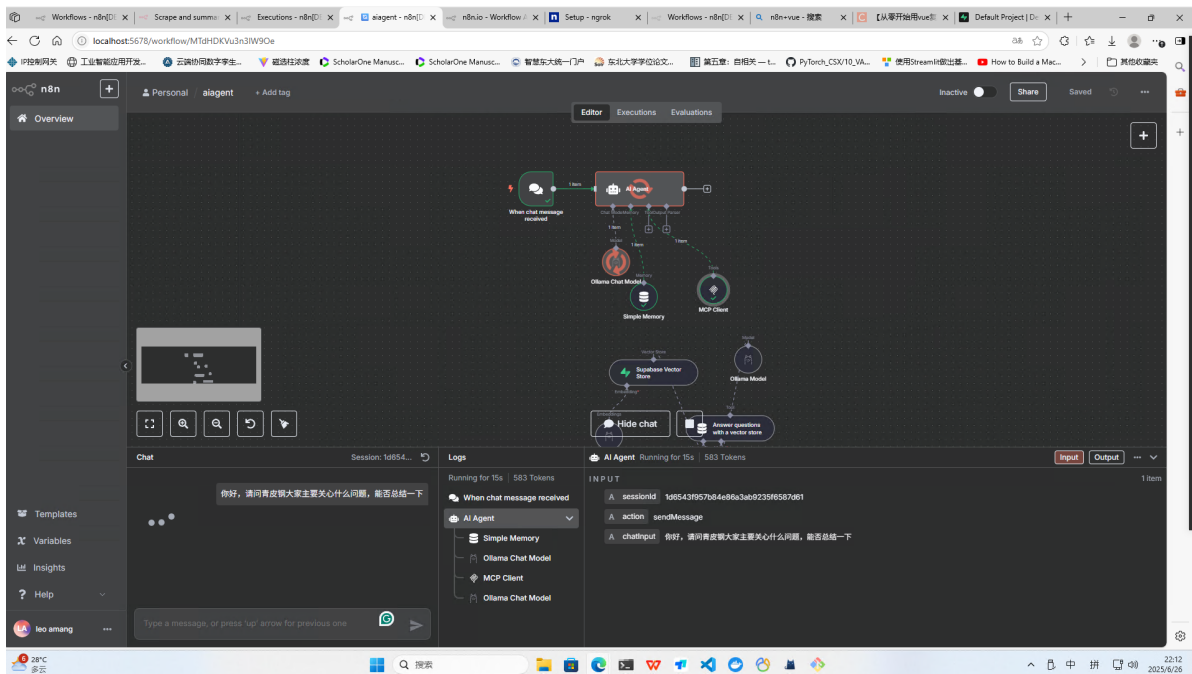
通过连接2.2中的MCP Server Trigger的sse生产url，实现交互



2.4具体流程

上述连接完成后开启n8n服务

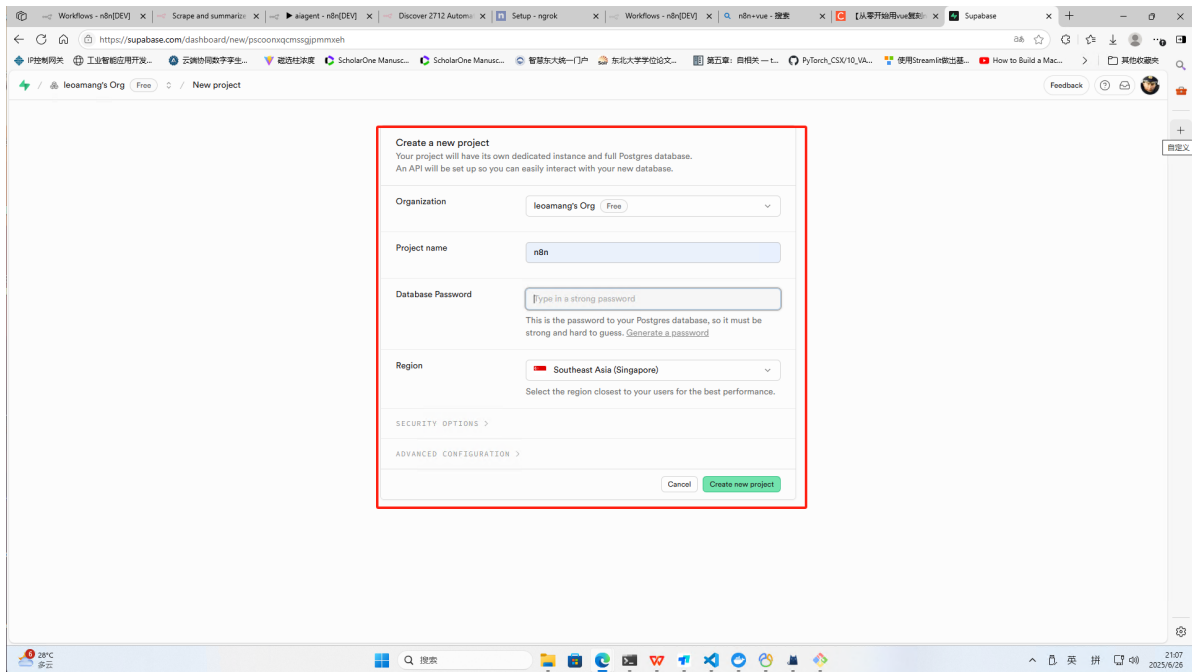




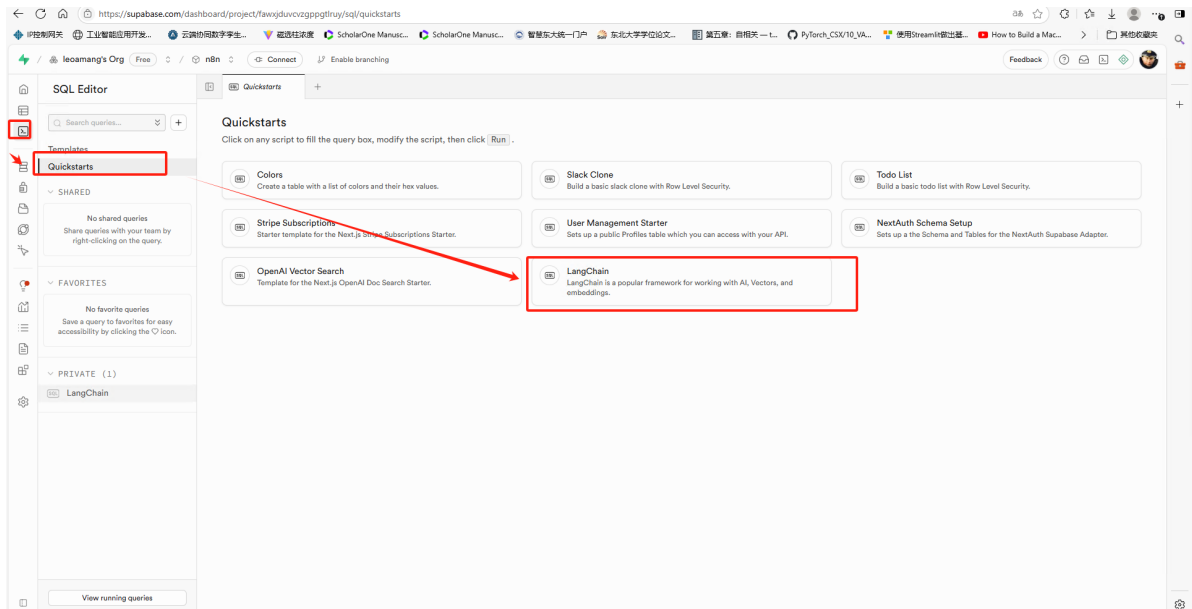
Supabase实现向量数据库

云服务<https://supabase.com/>

1. 创建账号并登录→创建一个新的project，命名为n8n，region选一个距离自己最近的地区→进入项目。

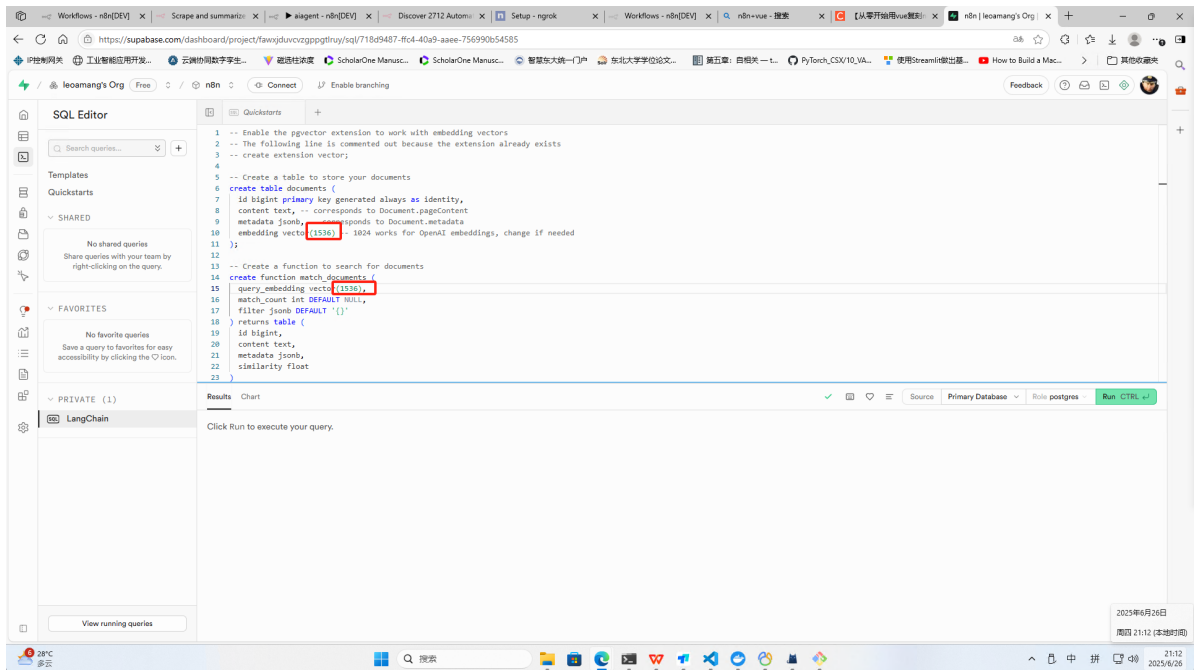


2. 点击侧边栏SQL Editor→选择Quickstarts→选择Langchain→会出现自动创建数据库的命令
langchain自动帮我们创建一套符合向量数据库规范的表结构

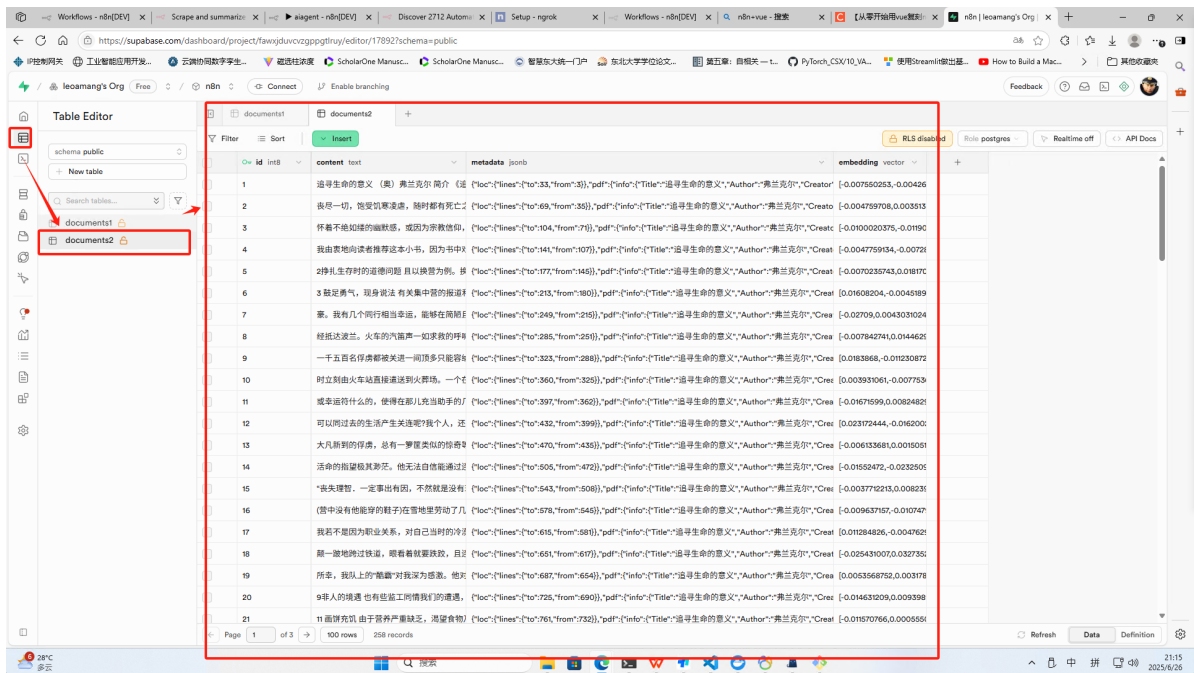
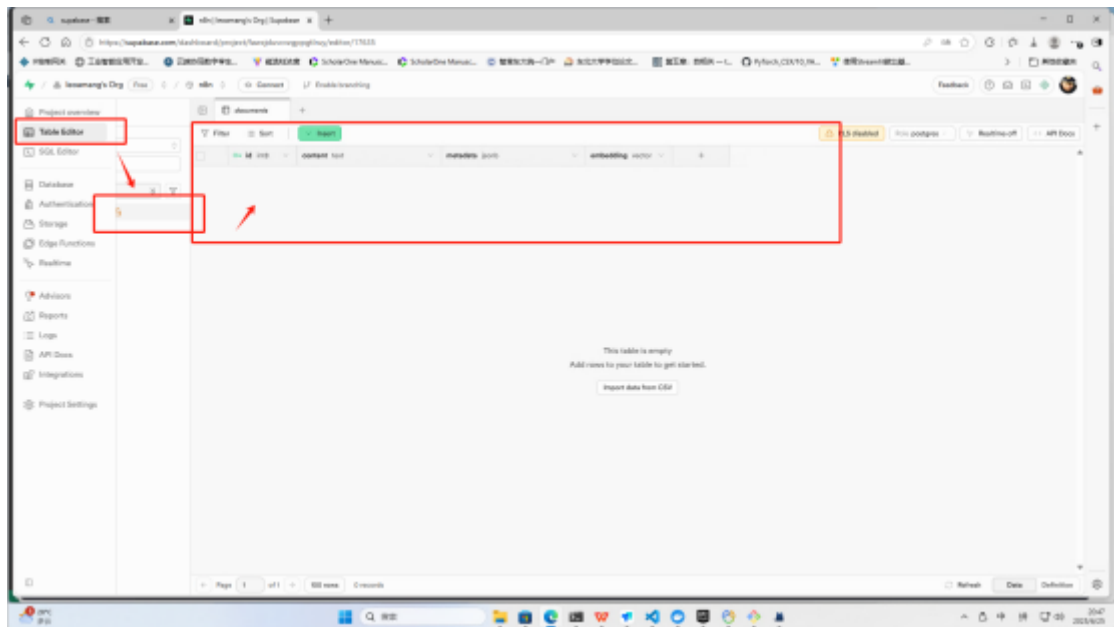


3. 使用时只用修改下图红色框里面的两个数字与向量大模型的输入维度匹配→修改完成后点击右边的绿色按钮run即可，创建数据库所需要的字段都创建好了

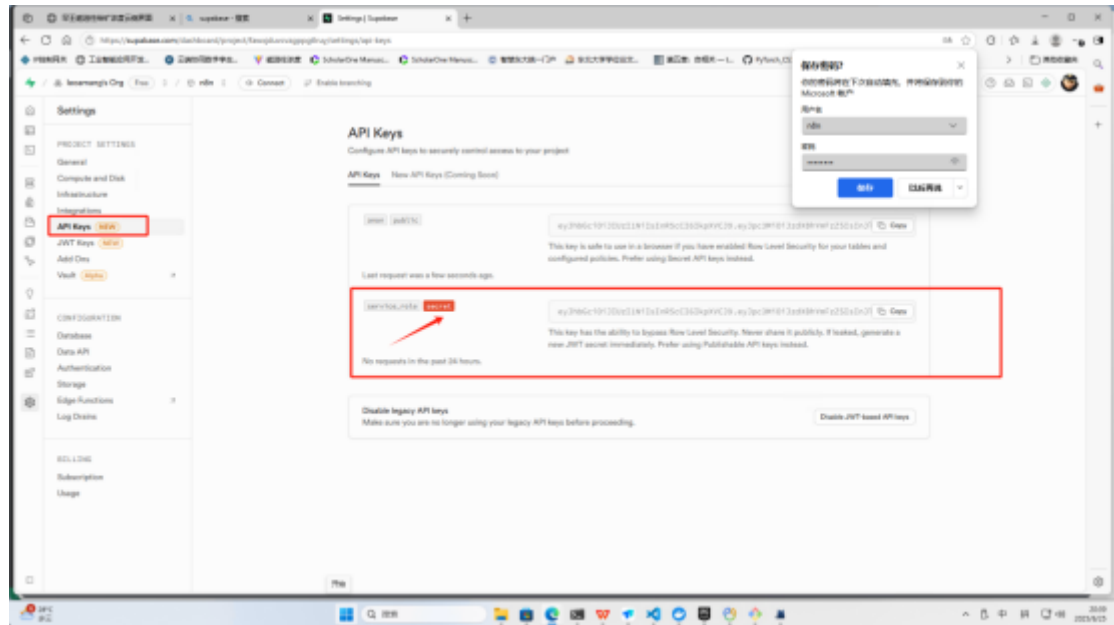
Qwen3是4096；openai是1536，Bert是768，shaw/dmeta-embedding-zh:latest是768



4. 进一步点击Table Editor查看创建的documents里面存储的向量

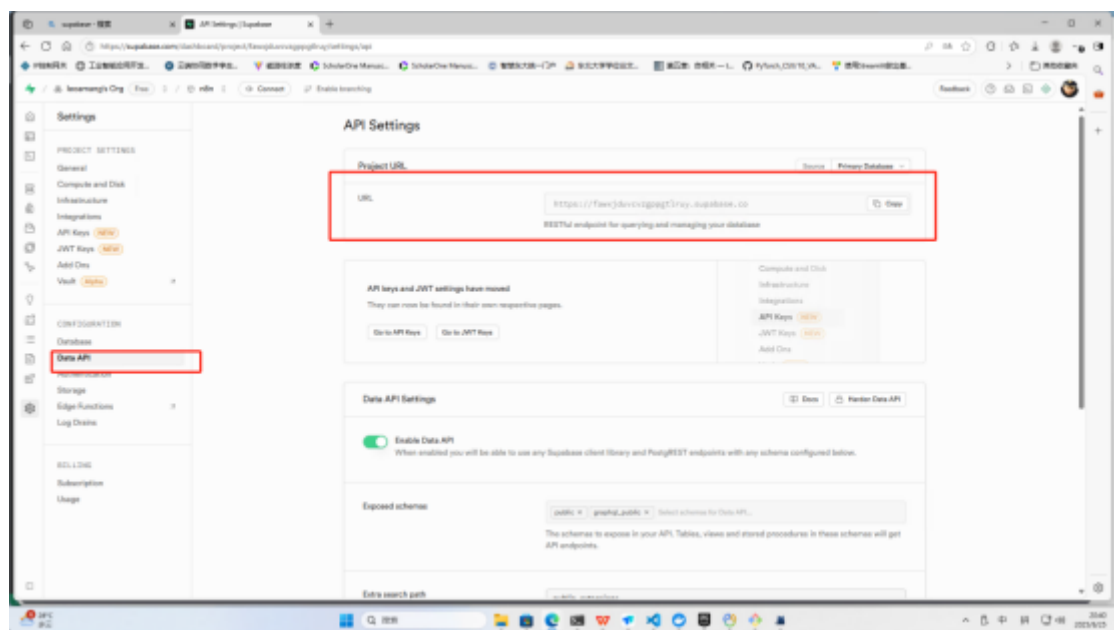


5. 点击设置→API keys→获取n8n中所需要的secret-key



6. 点击设置→Data API→获取n8n中所需要的url

<https://fawxjduvcvzgppgtlruy.supabase.co>



本地部署

Supabase 是一个开源的后端即服务（BaaS）平台，支持实时数据库、身份验证、文件存储等功能。以下是通过 Docker 和 CLI 两种方式在本地部署 Supabase 的详细步骤。

使用 Docker 部署

```
# 从github仓库克隆代码
git clone https://github.com/supabase/supabase.git
# 创建一个项目
mkdir supabase-project
# 等于说此时文件夹下有两个文件夹
# |
# |-supabase
# |-supabase-project
# 将原github 里面的docker文件夹复制到新的项目下
cp -rf supabase/docker/* supabase-project
```



```
# 将原github 里面的docker文件夹下的.env.example 复制到新的项目下成为.env文件
cp supabase/docker/.env.example supabase-project/.env
# 进入supabase-project
cd supabase-project
# 拉取镜像
docker compose pull
# 启动容器运行镜像
docker compose up -d

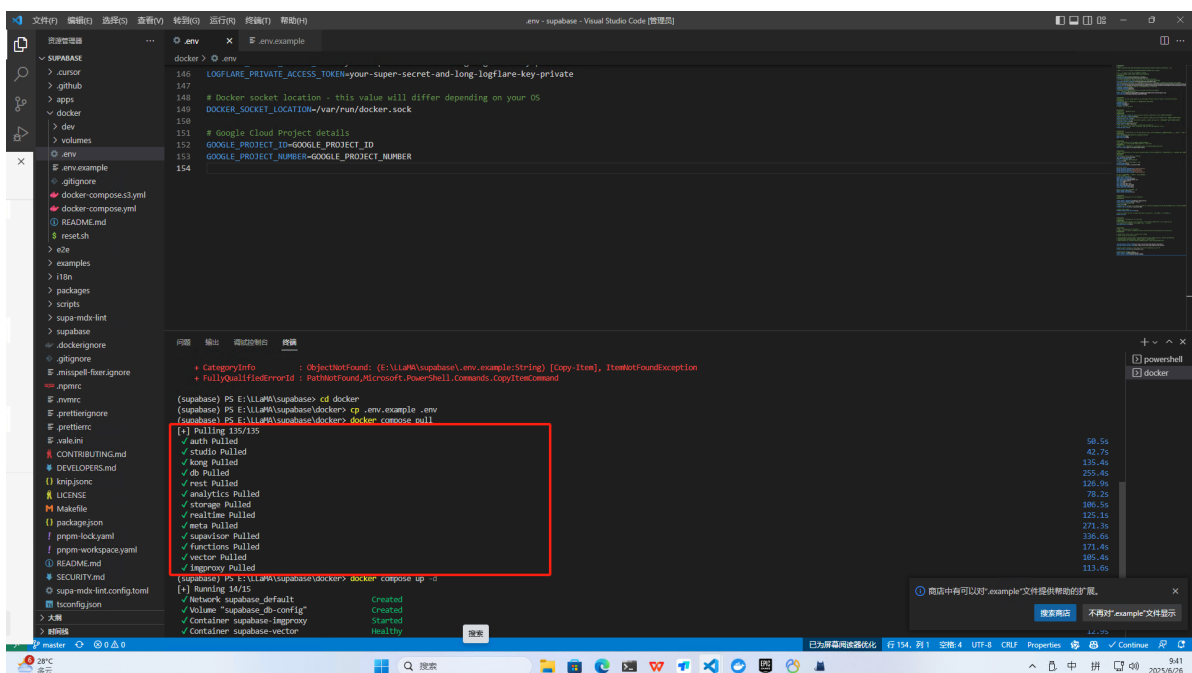
# 确保所有容器正常运行，可通过 docker ps 查看状态。
docker ps
# 重启项目可以用
docker compose down
```

```
Administrator@PC-202212061551 MINGW64 /e/LLaMA
$ git clone https://github.com/supabase/supabase.git
Cloning into 'supabase'...
fatal: unable to access 'https://github.com/supabase/supabase.git/': Failed to connect to github.com port 443 after 21128 ms: Could not connect to server

Administrator@PC-202212061551 MINGW64 /e/LLaMA
$ ^C

Administrator@PC-202212061551 MINGW64 /e/LLaMA
$ git clone https://github.com/supabase/supabase.git
Cloning into 'supabase'...
remote: Enumerating objects: 370464, done.
remote: Counting objects: 100% (601/601), done.
remote: Compressing objects: 100% (260/260), done.
remote: Total 370464 (delta 444), reused 373 (delta 340), pack-reused 369863 (from 2)
Receiving objects: 100% (370464/370464), 1.63 GiB | 4.82 MiB/s, done.
Resolving deltas: 100% (262866/262866), done.
Updating files: 100% (12090/12090), done.

Administrator@PC-202212061551 MINGW64 /e/LLaMA
$ |
```



2. 访问Supabase

<http://localhost:8000/>

3. 此时env文件里面的东西都是默认的，需要修改 .env 文件中的默认配置：

```
JWT_SECRET=
ANON_KEY=
SERVICE_ROLE_KEY=
DASHBOARD_USERNAME=supabase
DASHBOARD_PASSWORD=this_password_is_insecure_and_should_be_updated
```

jwt和下面的两个token从下面的网址得到：[Self-Hosting with Docker | Supabase Docs](#)

4. 配置完成后后续的操作和云部署的一样，建立向量数据库

部署bug

- 用npm报错，猜测是版本的问题，所以用docker
- error: unexpected token: carriage return (column 4, code point U+000D) | 30 | end | ^ |
nofile:30:4 (elixir 1.18.2) lib/code.ex:571: Code.validated_eval_string/3

删除./Volumes/pooler/pooler.exs回车end后面的** (ErlangError) Erlang error: {:badarg, {~c"aead.c", 90}, ~c"Unknown cipher or invalid key size"}: * 1st argument: Unknown cipher or invalid key size

- JWT_SECRET 和 VAULT_ENC_KEY至少JWT不是下面的代码生成的，而是官网就有的。

openssl rand -hex 32 生成32字节即64字符的 Hex 密钥

- 使用代码自动生成向量数据库有时候会报错，更新代码

```
-- Enable the pgvector extension to work with embedding vectors
create extension vector;

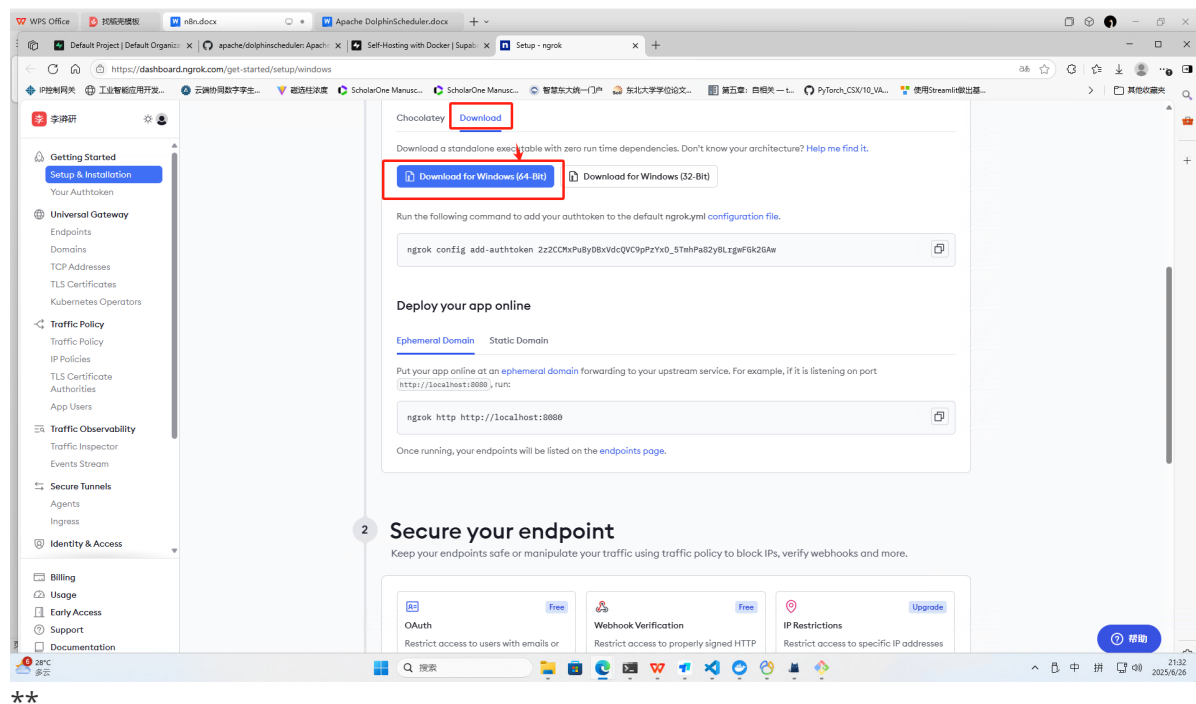
-- Create a table to store your documents
create table documents1 (
  id bigserial primary key,
  content text, -- corresponds to Document.pageContent
  metadata jsonb, -- corresponds to Document.metadata
  embedding vector(768) -- 1536 works for OpenAI embeddings, change if needed
);

-- Create a function to search for documents
create function match_documents1 (
  query_embedding vector(768),
  match_count int default null,
  filter jsonb DEFAULT '{}'
) returns table (
  id bigint,
  content text,
  metadata jsonb,
  similarity float
)
language plpgsql
as $$
#variable_conflict use_column
begin
  return query
  select
    id,
```

```
content,  
metadata,  
1 - (documents.embedding <=> query_embedding) as similarity  
from documents  
where metadata @> filter  
order by documents.embedding <=> query_embedding  
limit match_count;  
end;  
$$;
```

Ngrok本地映射公网

[ngrok - Online in One Line](https://dashboard.ngrok.com/get-started/online)



从网址上下载压缩包并解压→运行本地的exe文件→然后输入下面的代码

```
ngrok config add-authtoken 2z2CCMxPuByDBxVdcQVC9pPzYxO_5TmhPa82yBLrgwFGk2GAW  
ngrok http http://localhost:5678
```

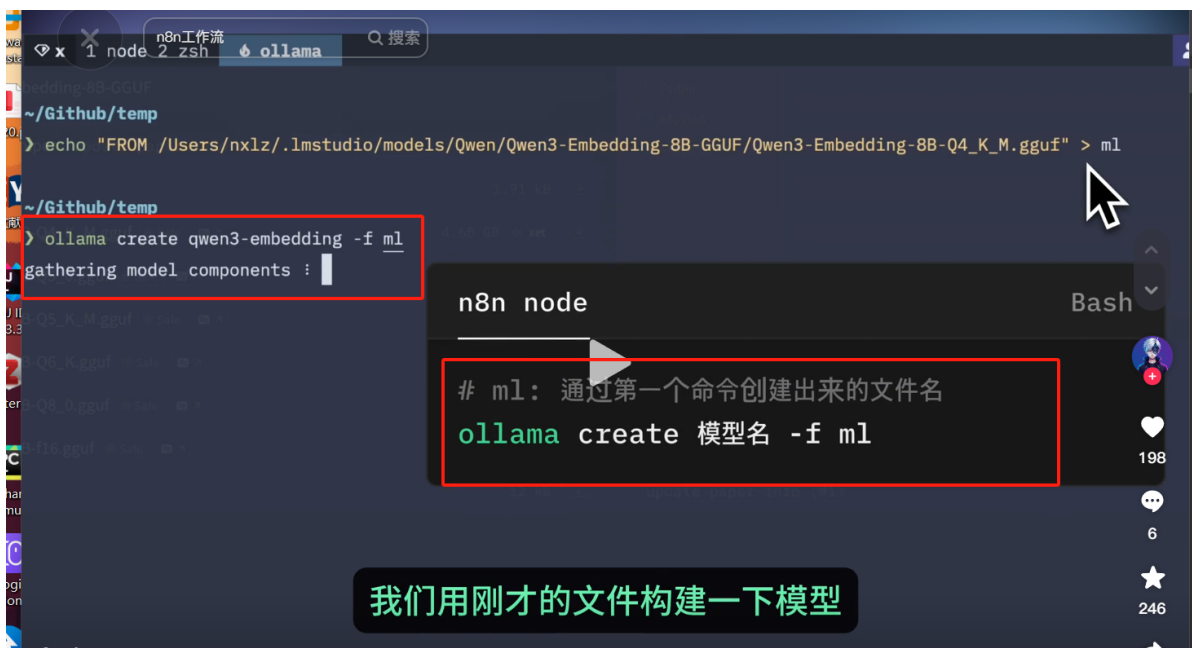
Ollama下载向量化模型

因为huggingface的出错了

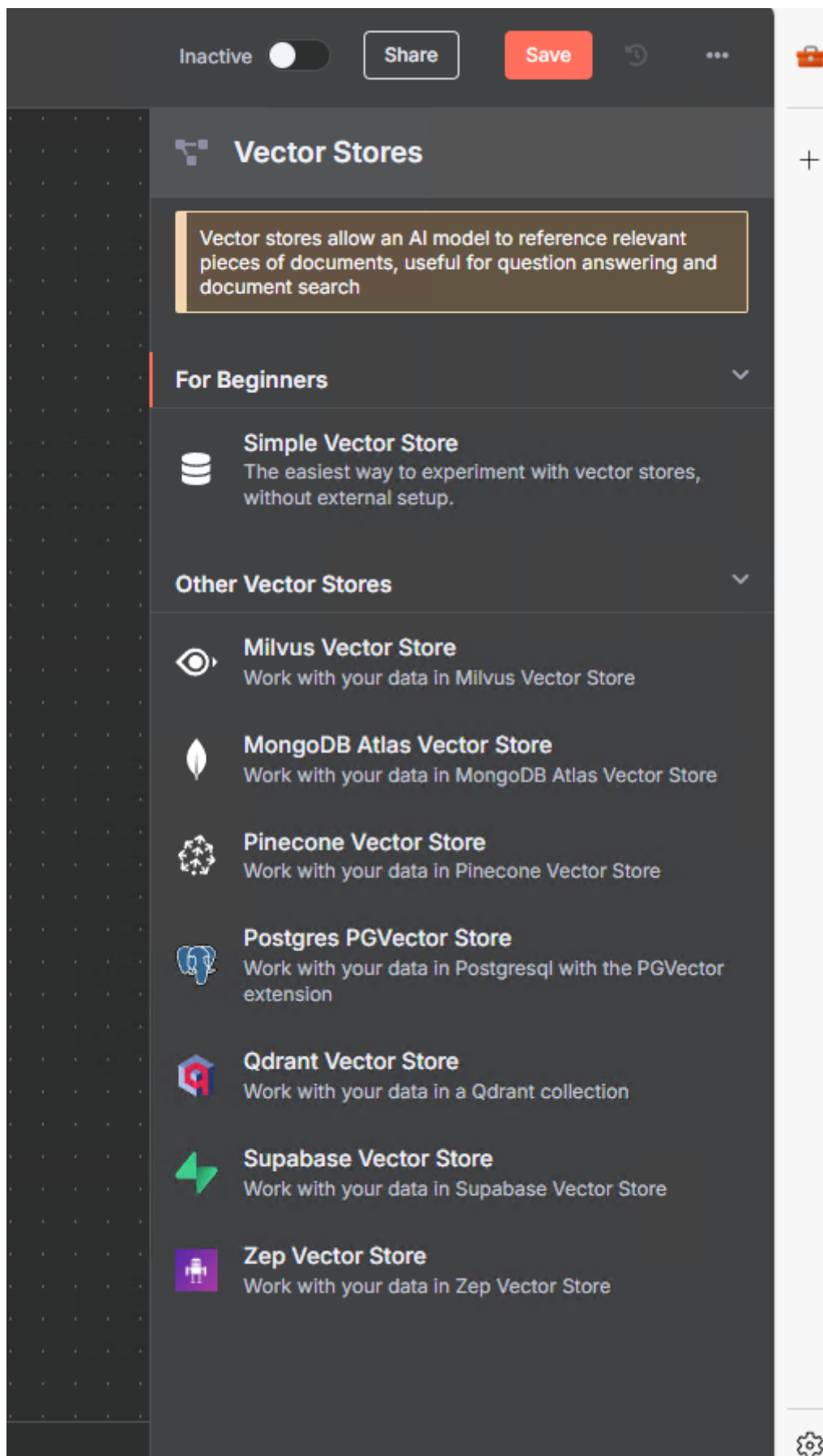
```
ollama pull hf.co/Qwen/Qwen3-Embedding-8B-GGUF:Q8_0
```

或者从huggingface下载完模型后

```
echo "FROM 模型地址" > 文件名  
ollama create 模型名 -f ml
```



N8N支持的向量数据库



注：使用docker部署的地址localhost映射为host.docker.internal

